



FlexiMigrate: enhancing live container migration in heterogeneous computing environments

Seyed Hossein Ahmadpanah¹ · Meghdad Mirabi² · Amir Sahafi¹ · Seyed Hossein Erfani¹

Received: 29 October 2024 / Revised: 27 April 2025 / Accepted: 2 June 2025 / Published online: 19 September 2025
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

This paper introduces FlexiMigrate, a novel framework designed to address the challenges of live container migration in heterogeneous computing environments. FlexiMigrate offers solutions to orchestration platform dependency and heterogeneity, migration overhead, network disruptions, and suboptimal migration decision-making. The framework utilizes a nested container architecture that decouples application containers from orchestration platforms, ensuring cross-platform compatibility. It also incorporates an advanced checkpointing technique to reduce migration overhead, Software-Defined Networking (SDN) integration to maintain continuous connectivity during migration, and a machine learning (ML)-based decision engine to optimize the migration process. These features enhance migration efficiency, improve resource utilization, and minimize service downtime. Experimental results show that FlexiMigrate introduces only a minor overhead compared to standard deployment while reducing total migration time by up to 46.2%, downtime by up to 78.8%, network overhead by up to 50%, average CPU utilization by up to 27.1%, and peak memory consumption by up to 22.4%. These findings validate FlexiMigrate's effectiveness in addressing container migration challenges in cloud and edge environments, offering a robust solution for real-world applications.

Keywords Live container migration · Nested container architecture · Orchestration platform compatibility · Software-defined networking (SDN) · Machine learning (ML) optimization

1 Introduction

1.1 Research motivation

Cloud computing and edge computing have revolutionized modern computing paradigms, providing organizations with scalable, cost-efficient, and low-latency computational resources [1–3]. Cloud computing enables on-demand access

to a shared pool of configurable resources, such as virtual machines, storage, and applications, facilitating operational scalability while reducing infrastructure costs [4–6]. Meanwhile, edge computing processes data closer to its source, reducing latency and network bandwidth consumption, which is particularly critical for time-sensitive applications such as autonomous systems and industrial IoT [7–10].

With the increasing adoption of multi-cloud and edge computing infrastructures, containerization has emerged as the de facto technology for deploying and managing applications in distributed environments. Containers offer portability by encapsulating applications along with their dependencies, ensuring consistent execution across different cloud platforms and edge devices [11–13]. This has led to a significant rise in container adoption, with the global container market projected to reach approximately US\$3.6 billion by 2026, growing at a 22% Compound Annual Growth Rate (CAGR) from 2021¹.

However, managing containers across diverse infrastructures remains a challenge due to dependencies on container

✉ Meghdad Mirabi
meghdad.mirabi@cs.tu-darmstadt.de

✉ Amir Sahafi
sahafi@iau.ac.ir
Seyed Hossein Ahmadpanah
h.ahmadpanah@iau.ac.ir
Seyed Hossein Erfani
h_erfani@azad.ac.ir

¹ Department of Computer Engineering, ST.C., Islamic Azad University, Tehran, Iran

² Faculty of Computer Science, Technical University of Darmstadt, Darmstadt, Germany

¹ <https://www.marketsandmarkets.com/Market-Reports/container-security-market-156059467.html>.

orchestration platforms such as Kubernetes² (K8s), OpenShift³, Tanzu⁴, and K3s⁵. Although Kubernetes (K8s) dominates container orchestration, many enterprises use customized or alternative solutions, creating interoperability challenges. This dependency raises concerns about vendor lock-in, portability constraints, and operational complexity when migrating workloads across different platforms.

Moreover, the heterogeneity in orchestration platforms (e.g., Kubernetes (K8s), Docker Swarm⁶) and network/cloud-edge infrastructures further complicates container migration. This diversity introduces additional challenges in maintaining consistent performance and ensuring seamless migration across cloud and edge environments. To address these challenges, a flexible and efficient migration framework is essential, one that can operate seamlessly across a variety of cloud and edge computing environments.

1.2 Research problem

Live container migration is essential for ensuring workload continuity, resource optimization, and fault tolerance in distributed computing environments [14–18]. It enables active container instances to be transferred across cloud and edge nodes with minimal disruption. However, several key challenges impede the efficient live migration of containers in heterogeneous cloud-edge infrastructures:

- *Orchestration Platform Dependency*: Many containerized applications are tightly coupled with Kubernetes (K8s) as their orchestration framework. Although Kubernetes (K8s) is the de facto standard, significant interoperability challenges arise when migrating across competing platforms such as Docker Swarm, Nomad, and Apache Mesos, which use different APIs, network models, and state management strategies. For example, the ProKube framework [19], which relies on Kubernetes-specific APIs and state management, cannot migrate containers to Docker Swarm clusters. Consequently, cross-platform migration often requires modifications to container configurations, leading to portability issues [20, 21].
- *Orchestration Platform Heterogeneity*: Modern cloud and edge infrastructures often consist of multiple orchestration platforms running simultaneously, such as Kubernetes (K8s), OpenShift, and Docker Swarm. These platforms have distinct scheduling policies, networking configurations, and security models, making

seamless migration across them highly challenging. The lack of standardized orchestration-layer abstractions results in inconsistencies in resource allocation, service discovery, and container runtime compatibility [22, 23].

- *State Transfer Overhead*: Traditional live migration techniques transfer the full state of containers, including memory, storage, and network connections. These techniques often result in prolonged downtime and high data transfer costs, which are particularly challenging in resource-constrained cloud-edge environments [24, 25]. Thus, developing more efficient migration strategies is crucial to reducing data transfer overhead, minimizing service interruptions, and ensuring seamless application execution across distributed infrastructures.
- *Network Disruptions and Service Downtime*: Migration events can lead to temporary network disruptions due to IP address reassignments, DNS propagation delays, and session inconsistencies. For stateful applications and latency-sensitive services, even short disruptions can cause significant performance issues, affecting end-user experience and application reliability [26, 27]. Advanced network management solutions are necessary to mitigate these disruptions.
- *Suboptimal Migration Decision-Making*: Existing migration strategies often rely on static policies or basic heuristics that fail to adapt to dynamic cloud-edge environments. Inefficient migration decisions may lead to resource underutilization, increased operational costs, or degraded service quality [28, 29]. Intelligent migration strategies leveraging machine learning or optimization-based approaches could enhance decision-making, improving the selection of migration destinations, balancing workload distribution, and minimizing performance overhead.

1.3 Research scope

The primary focus of this research is to address the challenges associated with live container migration in heterogeneous cloud and edge environments. Specifically, this research explores the orchestration platform heterogeneity and platform dependency, which are significant challenges when migrating containers across different platforms such as Kubernetes (K8s), Docker Swarm, and OpenShift. The research aims to provide a seamless migration solution that minimizes migration overheads, ensures continuous connectivity, and adapts intelligently to resource and workload variations. Techniques like advanced checkpointing, nested container architecture, Software-Defined Networking (SDN), and machine learning-based (ML-based) decision-making are employed to achieve these objectives, ensuring that FlexiMigrate adapts to changing conditions and reduces migration overheads.

² <https://kubernetes.io/>.

³ <https://openshift.com/>.

⁴ <https://www.vmware.com/products/app-platform/tanzu>.

⁵ <https://k3s.io/>.

⁶ <https://docs.docker.com/engine/swarm/>.

However, certain factors are beyond the scope of this research. While this work addresses orchestration platform heterogeneity, it does not consider hardware/software heterogeneity, such as variations in CPU architectures, OS versions, or kernel configurations. Support for cross-platform portability related to these variations, such as multi-architecture container images (e.g., ARM/x86), is also excluded. Furthermore, although FlexiMigrate leverages advanced checkpointing, a nested container architecture, an SDN-based approach, and ML techniques to enhance migration efficiency, their applicability beyond container migration or to diverse hardware platforms is not explored. The scope of these techniques is thus confined to orchestrator and network heterogeneity contexts.

1.4 Research contributions

This paper makes several key contributions to the field of container migration, particularly in cloud and edge computing environments:

- We introduce FlexiMigrate, a novel framework for live container migration that offers innovative solutions to several migration challenges. It includes:
 - *Orchestrator-Agnostic Migration*: FlexiMigrate minimizes dependence on specific container orchestration platforms (e.g., Kubernetes (K8s), Docker Swarm, OpenShift) by employing a nested container architecture. Rather than directly managing application containers, FlexiMigrate encapsulates them within a specially designed parent container. This parent container acts as an intermediary, abstracting away orchestration-specific configurations, APIs, and scheduling mechanisms. By decoupling the application containers from the underlying orchestration framework, FlexiMigrate enables seamless migration across different orchestration platforms without requiring modifications to the application containers themselves. This approach ensures that the core application remains unaware of the migration process, preserving portability, and mitigating vendor lock-in issues. This solution directly addresses challenges related to orchestration platform dependency and heterogeneity.
 - *Efficient State Transfer Mechanism*: FlexiMigrate incorporates an advanced checkpointing technique that transfers only the incremental state changes of containers instead of full snapshots. By adopting this incremental transfer approach, FlexiMigrate significantly reduces the amount of data to be migrated, minimizing the migration overhead. This approach ensures
- reduced service interruptions, especially in resource-constrained cloud-edge environments, improving overall system performance and operational efficiency.
- *Resilient Network Adaptation*: FlexiMigrate integrates with an SDN approach to enable real-time reconfiguration of the network infrastructure, ensuring seamless connectivity during the migration process. By dynamically adjusting network paths and handling IP address reassignments, DNS propagation, and session persistence, FlexiMigrate minimizes service downtime and prevents session disruptions that typically occur during migration. This feature is crucial for stateful applications and latency-sensitive services, ensuring minimal impact on user experience and application performance during the migration process.
- *Intelligent Migration Decision-Making*: FlexiMigrate leverages ML techniques for intelligent migration decision-making. By continuously analyzing real-time system conditions, resource availability, and workload distribution, it optimizes migration decisions to enhance resource utilization, reduce migration overhead, and prevent service degradation. The ML-based decision-making mechanism enables FlexiMigrate to dynamically adapt to changing conditions, effectively addressing the challenge of suboptimal migration decisions.
- We provide a comprehensive evaluation of FlexiMigrate through extensive experiments, demonstrating its effectiveness in improving migration efficiency, minimizing downtime, and enhancing resource utilization in heterogeneous distributed environments. Additionally, we assess FlexiMigrate's novel nested container architecture, highlighting how it enables seamless, orchestrator-agnostic migrations with minimal overhead and high service availability.

The rest of the paper is organized as follows: Sect. 2 provides an overview of related work on container migration and compares it with the FlexiMigrate framework. Section 3 describes the FlexiMigrate framework, detailing its system architecture, explaining its nested container architecture, and describing its SDN-based network management for efficient container migration. Section 4 presents a detailed description of the components and modules of the FlexiMigrate framework. Section 5 evaluates the performance of FlexiMigrate through a series of experiments. Section 6 provides an explanation of FlexiMigrate's functionalities and their relation with performance improvements in container migration, and the challenges of implementing FlexiMigrate in a multi-container environment. Finally, Sect. 7 concludes the paper with suggestions for future research.

2 Related work

Numerous surveys have thoroughly analyzed container migration techniques and the challenges associated with them. For instance, an extensive review of live migration strategies for both virtual machines and containers in cloud computing environments is presented in [14]. This review discusses pre-copy, post-copy, and hybrid techniques, evaluating trade-offs in migration time, service downtime, and resource utilization. It also highlights unique challenges in container migration, such as the demand for lightweight checkpointing and efficient state transfer methods, which distinguish it from virtual machine migration.

Research focused on edge computing, such as in [30], surveys container orchestration and migration techniques tailored specifically for edge environments. This work emphasizes the importance of network heterogeneity, resource constraints, and mobility. The authors stress the need for adaptive migration policies that respond dynamically to network conditions and workload variations at the edge. Similarly, a systematic review [31] examines container-based cloud and edge architectures, analyzing various migration strategies while highlighting the growing importance of multi-cloud and hybrid cloud-edge environments in supporting complex distributed infrastructures.

Other studies examine migration's role in specialized distributed systems. For example, in fog computing, the authors in [32] explore the impact of container migration on load balancing, fault tolerance, and service continuity. They also identify gaps in managing diverse hardware and software setups commonly found in fog and edge contexts. Security aspects of container migration are addressed in [9], where a literature review on the risks of live migration, such as data confidentiality and checkpoint vulnerabilities, is presented. The study emphasizes the need for secure migration protocols in multi-tenant cloud environments.

Several frameworks provide innovative solutions for container migration in specific environments. Voyager [33] extends the Checkpoint/Restore in Userspace (CRIU) mechanism to facilitate seamless migration across hosts by addressing network state and file system consistency. While it improves service downtime and migration efficiency, Voyager's reliance on CRIU limits its applicability across diverse container runtimes. Similarly, LIMOCE [34] employs a pre-copy mechanism and a container layering approach to reduce service downtime in edge environments. In [35], the authors modify the Linux kernel and implement LXD, which uses liblxc to launch containers, with migration facilitated using CRIU's checkpoint/restore mechanism. Although this approach is more efficient than pre-copy VM migration approaches, its applicability in cloud-edge hybrid systems may be limited due to specific constraints in edge scenarios.

Additional work focuses on migration techniques aimed at resource optimization. The framework in [36] introduces a

three-phase approach to minimize service downtime through efficient resource management during virtual machine live migration. While effective for virtual machines, it may not be suitable for containers, which experience rapid state changes or unstable network conditions. In [37], the authors propose a memory prediction mechanism to reduce migration memory transfer, although this solution adds computational overhead, potentially impacting resource-constrained environments. The framework in [38] proposes a machine learning-based approach (VMLM) that uses a two-phase process—a real-time migration stage and a supervised learning model trained on VM parameters (such as CPU utilization and memory allocation)—to dynamically optimize migration decisions. By combining runtime resource management with predictive analytics, this study improves virtual machine migration techniques and provides a scalable solution for contemporary cloud infrastructures. In [39], a two-objective optimization model is developed to comprehensively address the container migration problem in edge computing. The model is formulated as a constrained optimization problem, effectively balancing the trade-off between reducing latency and enhancing reliability using the improved binary whale optimization algorithm.

Mechanisms such as checkpointing and buffering have been proposed to enhance migration reliability, as seen in [40], though bandwidth requirements may limit their applicability in resource-scarce environments. Additionally, the solution proposed in [41] introduces vertical elasticity for containers, dynamically scaling resources based on workload to reduce migration downtime and operational costs. However, adapting this approach to multi-cloud environments remains a challenge.

Specific to Kubernetes (K8s), a pod migration approach is introduced in [42] to enable live migration across clusters, which is beneficial for stateful and fault-tolerant systems. However, the proposed approach's focus may limit its applicability across heterogeneous platforms. Similarly, in [43], live service migration in mobile edge environments is optimized for clusters of mobile devices, although the approach exhibits weaknesses in scalability under dynamic conditions. Additionally, the ProKube framework [19] is introduced as a proactive container scaling and migration orchestrator for Kubernetes (K8s) in heterogeneous edge environments. Designed for latency-sensitive machine learning applications, ProKube uses performance logs to make scaling and migration decisions, reducing unnecessary migrations and scaling operations while meeting latency service level agreements (SLAs).

Further innovations include container-layered storage in [44], which minimizes data transfer in edge scenarios by leveraging layered storage. However, added complexity may hinder synchronization across diverse edge nodes. In [45], the problem of container-based service migration in edge computing is investigated to address mobility-aware migration. Additionally, the Ubiquitous Migration Solution (UMS) [46]

enables transparent migration across autonomous systems with heterogeneous orchestrators. While UMS offers diverse strategies with low overhead, it faces limitations in high-security environments or those requiring precise migration control.

Although containerized environments are the focus of this research, alternate migration paradigms are emerging with new workload abstractions such as *WebAssembly* (Wasm). Wasm [47] is well-suited for edge and serverless contexts because of its lightweight, portable binaries, which allow for near-native execution across diverse platforms with minimal overhead. Similar to the CRIU mechanism for containers, recent initiatives have explored stateful Wasm migration via checkpointing (e.g., [48]).

Table 1 presents a comparative analysis of existing container migration approaches based on platform independence, migration techniques, resource optimization methods, dynamic network management capability, machine learning support, multi-container implementation, and edge-specific adaptations. Many existing approaches face limitations such as strong platform dependency, lack of dynamic network management, suboptimal resource utilization, and inadequate support for multi-container and edge computing environments. In contrast, FlexiMigrate

introduces a platform-independent, orchestrator-agnostic framework designed for seamless operation across diverse cloud and edge environments. By integrating a nested container architecture, SDN-based network management, and machine learning-driven optimization, FlexiMigrate reduces migration overhead and enhances adaptability across various orchestration platforms. These innovations represent a significant advancement in flexible and efficient container migration, addressing key challenges in distributed computing environments.

3 FlexiMigrate framework

This section provides an overview of the FlexiMigrate framework, detailing its system architecture, explaining its nested container architecture, and describing its SDN-based network management for efficient container migration.

3.1 Framework overview

Figure 1 illustrates the placement of the FlexiMigrate framework between two cloud/edge infrastructures, highlighting its

Table 1 Comparison of Container Migration Approaches

Approach	Platform Independence	Migration Technique	Resource Optimization Method (R)	Dynamic Network Management Capability	Machine Learning Support (M)	Multi-Container Implementation	Edge-Specific Adaptation*
FlexiMigrate	✓	Nested Container	R1 & R2	✓	M1	✓	✓
Approach in [33]	×	CRIU-based	×	×	×	×	×
Approach in [34]	×	Pre-copy & Container Layering	R3	×	×	×	✓
Approach in [35]	×	LXD/CRIU Migration	R10	×	×	×	×
Approach in [36]	×	LXD/CR Migration	R4	×	×	×	×
Approach in [37]	×	Dirty Memory Prediction	R5	×	M2	×	×
Approach in [38]	×	ML-Driven VM Migration	R11	×	M5	×	×
Approach in [39]	×	Dual-Objective Optimization	R9	✓	M4	✓**	✓
Approach in [40]	×	Checkpointing & Buffering	×	×	×	×	×
Approach in [41]	×	Vertical Elasticity	R6	×	×	×	×
Approach in [42]	×	Pod Migration	×	×	×	✓	×
Approach in [43]	×	Layering Migration	×	×	×	×	✓
Approach in [19]	×	Container Scaling	R8	✓	M3	✓	✓
Approach in [44]	×	Layered Storage	R7	×	×	×	✓
Approach in [45]	✓	Mobility-Awareness	×	✓	×	×	×
Approach in [46]	✓	Ubiquitous Migration	×	✓	×	✓	×
Approach in [47]	✓	Stateful Wasm Migration	R10	✓	×	×	✓
Approach in [48]	✓	Stateful Wasm Migration	R10	✓	×	×	✓

Resource Optimization Methods: R1 = Memory Deduplication, R2 = Bandwidth Control, R3 = Layered Storage Optimization, R4 = Three-Phase Memory Management, R5 = Memory Prediction, R6 = Dynamic Resource Scaling, R7 = Storage Deduplication, R8 = Latency-Aware Scheduling, R9 = Latency & Reliability Optimization, R10 = Lightweight Execution, R11 = Energy Optimization.

Machine Learning Support: M1 = Random Forest, M2 = Statistical Modeling, M3 = Proactive Policy Learning, M4 = IBWOA Optimization, M5 = Supervised Learning (VMLM).

*Designed/tested for Constraints in Edge Computing: Resource-limited devices, WAN dynamics, or lightweight orchestration (e.g., K3s).

**Multi-Container Experiments up to 50 Containers

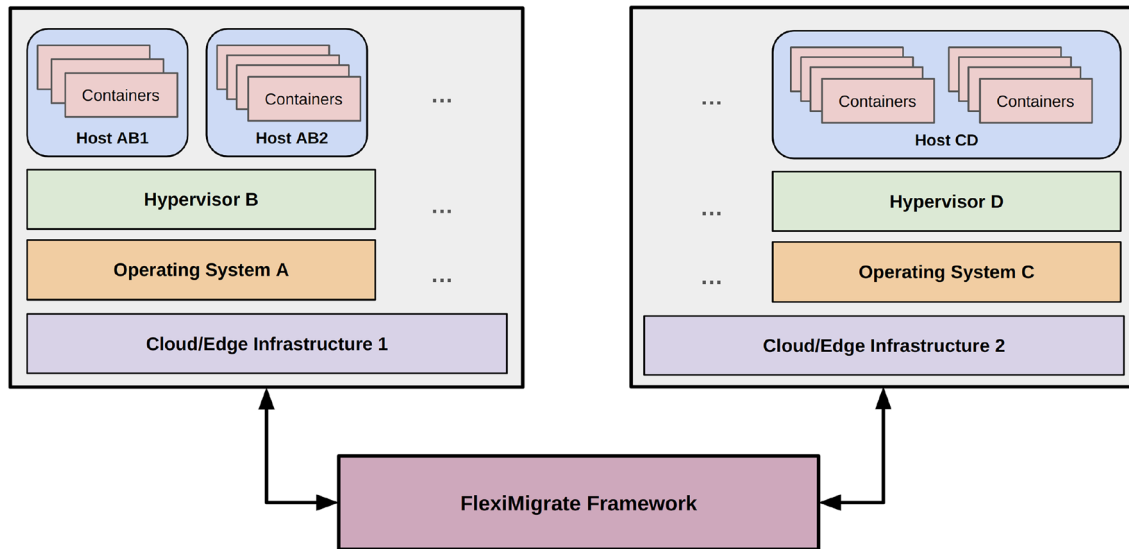


Fig. 1 FlexiMigrate Framework in Cloud/Edge Infrastructure for Live Container Migration

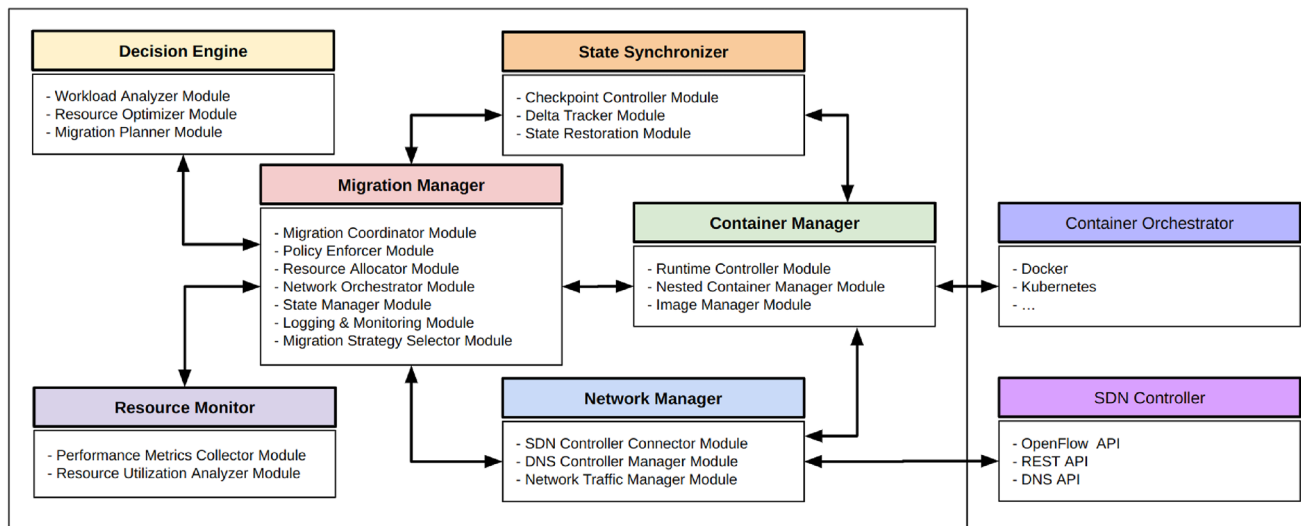


Fig. 2 Component and Module Architecture of the FlexiMigrate Framework

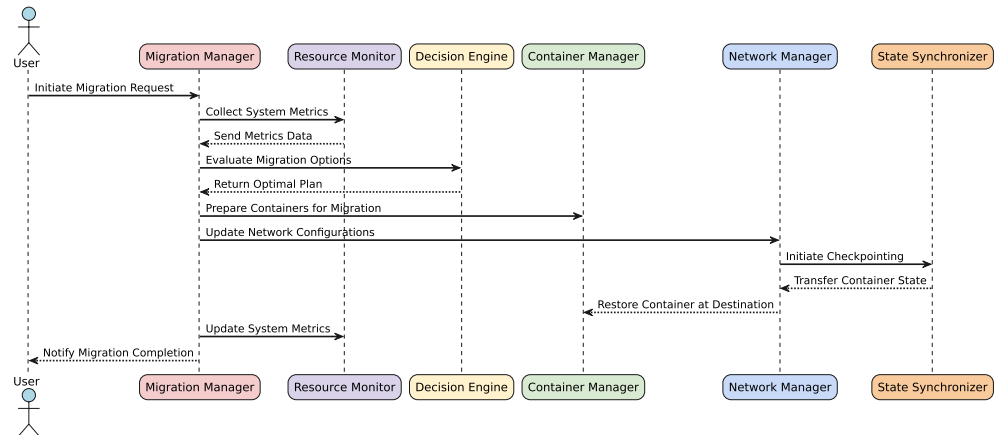
role in enabling seamless live container migration. At the top, the two separate cloud/edge infrastructures are depicted, while FlexiMigrate is positioned below, serving as a bridge between them. The framework orchestrates the movement of containerized applications across various infrastructures by abstracting and managing the complexity of container migration.

Figure 2 illustrates the overall architecture of the FlexiMigrate framework, which comprises several key components working in concert to enable flexible and efficient container migration across diverse cloud and edge environments. The framework includes the following components:

- **Migration Manager:** This central orchestrating component coordinates and executes container migrations across heterogeneous cloud and edge environments. It acts as

the primary interface between user-initiated migration requests and the various specialized components of the framework, providing an orchestrator-agnostic migration.

- **Resource Monitor:** This component continuously collects performance metrics and resource utilization data, analyzes them using a threshold-based mechanism, and provides real-time insights to facilitate appropriate migration decisions.
- **Decision Engine:** This component is responsible for workload analysis, resource optimization, and migration planning. By leveraging advanced machine learning techniques, it makes intelligent decisions that enhance system performance and overall efficiency.
- **Container Manager:** This component manages container lifecycle operations, facilitates seamless migrations, and ensures efficient image management across

Fig. 3 System Workflows for Live Container Migration in the FlexiMigrate Framework

diverse cloud and edge environments. It is responsible for implementing the nested container architecture in FlexiMigrate by decoupling the application containers from the underlying orchestration framework.

- **Network Manager:** It orchestrates network operations to ensure seamless migration and uninterrupted service availability. It leverages SDN principles and advanced network management techniques to handle the complex challenges in network management associated with live container migration in cloud and edge environments.
- **State Synchronizer:** This component ensures consistent and efficient state transfer during container migration by employing advanced techniques to capture, track, and restore container states across heterogeneous cloud and edge environments.

The components of FlexiMigrate are designed to operationalize its key contributions. The orchestrator-agnostic migration is realized through the *Migration Manager* component, enabling seamless migration across different platforms. The nested container architecture is implemented by the *Container Manager* component to address orchestration platform dependency and heterogeneity. The efficient state transfer mechanism is supported by the *State Synchronizer* component, which reduces migration overhead. Resilient network adaptation is achieved through SDN-driven network reconfiguration in the *Network Manager* component, ensuring minimal service disruption. Intelligent migration decision-making is facilitated by machine learning techniques used in the *Decision Engine* component, optimizing resource utilization and system performance. Together, these components collectively address the critical challenges of live container migration, making FlexiMigrate a comprehensive and adaptable solution for cloud and edge environments.

It is important to note that two components, *Container Orchestrator* and *SDN Controller*, are external to the FlexiMigrate framework. The *Container Orchestrator* component integrates FlexiMigrate with existing container platforms in cloud and edge infrastructures. The *SDN*

Controller component establishes a low-level connection between FlexiMigrate and the SDN network to manage and orchestrate network operations during container migration.

Figure 3 illustrates the system workflow for live container migration in the FlexiMigrate framework. As shown, the migration process begins when a user triggers a container migration request, initiating a series of coordinated actions. First, the *Migration Manager* component collects real-time system metrics from the *Resource Monitor* component, which are essential for making informed migration decisions. With this data in hand, the *Decision Engine* component evaluates various potential migration paths and identifies the optimal plan. Following this, the *Migration Manager* component works closely with the *Container Manager* component to prepare the necessary containers and collaborates with the *Network Manager* component to ensure that all network configurations are properly aligned. Once preparations are complete, the *Container Manager* component and the *State Synchronizer* component execute the migration, effectively transferring the container state to its new location. Finally, the *Migration Manager* component wraps up the process by updating the system metrics and notifying the user of the successful migration, ensuring a seamless transition.

3.2 Nested container architecture

In the FlexiMigrate framework, the *Container Manager* component implements the *Nested Container Manager* module, which introduces an innovative nested container architecture designed for transparent and minimally disruptive container migration. This architecture encapsulates application containers within migration-aware parent containers, leveraging Linux kernel features such as namespaces and cgroups to establish a hierarchical structure. The nested approach enhances isolation, facilitates flexible resource management, and simplifies state handling. Crucially, it enables the migration of an entire container stack without requiring modifications to the application container, ensuring seamless transitions while maintaining compatibility with existing container orchestration platforms.

The *Nested Container Manager* module is responsible for creating, managing, and migrating nested containers. It ensures consistency and isolation throughout the migration process, acting as a transparent layer between application containers and container orchestration platforms. By abstracting the migration process from the application layer, it allows for seamless integration with orchestration platforms without requiring any changes to the application containers.

In this module, the migration process is handled by the *MigrateContainer* algorithm (Algorithm 1).

the container is saved for migration. If the checkpointing process fails, the algorithm returns a FAILURE status. The container state is then transferred to the destination using the *TransferState* algorithm (Algorithm 4). Finally, the container's state is restored at the destination using the *Restore* algorithm (Algorithm 5), ensuring the container continues its operation seamlessly.

The *GetNestedContainer* algorithm (Algorithm 2) is invoked within the *MigrateContainer* algorithm (Algorithm 1) to retrieve the nested container from the parent container's metadata. This step is crucial because the nested con-

```

1: procedure MIGRATECONTAINER(container, destination)
    # Retrieve the nested container.
2:   nestedContainer ← GETNESTEDCONTAINER(container)
    # Initiate the checkpoint process.
3:   status ← CHECKPOINT(container, nestedContainer, destination)
    # If checkpointing fails, return FAILURE.
4:   if status == FAILURE then
5:     return FAILURE
6:   end if
    # Transfer the saved state data to the destination.
7:   TRANSFERSTATE(container, destination)
    # Restore the container and its nested container.
8:   status ← RESTORE(container, destination)
    # If restoration fails, return FAILURE.
9:   if status == FAILURE then
10:    return FAILURE
11:  end if
    # Return SUCCESS if all steps are completed successfully.
12:  return SUCCESS
13: end procedure

```

Algorithm 1 MigrateContainer Algorithm

The algorithm involves several steps, including retrieval of nested container, checkpointing, state transfer, and restoration of the container at the destination. First, the algorithm retrieves the nested container from the parent container using the *GetNestedContainer* algorithm (Algorithm 2). Then, it triggers the checkpointing process using the *Checkpoint* algorithm (Algorithm 3), which ensures the current state of

tainer is encapsulated within the parent container, and this algorithm facilitates accessing the actual container object required for migration. The algorithm first inspects the parent container's metadata to extract the ID of the nested container and then retrieves the nested container using this ID. This operation is essential before migration can proceed.

```

1: procedure GETNESTEDCONTAINER(container)
    # Use container runtime API to inspect the container.
2:   parent_metadata ← INSPECTCONTAINER(container)
    # Extract nested container ID from the parent container's metadata.
3:   nested_id ← GETNESTEDCONTAINERID(parent_metadata)
    # Return the nested container object using the nested container ID.
4:   return GETCONTAINER(nested_id)
5: end procedure

```

Algorithm 2 GetNestedContainer Algorithm

The *Checkpoint* algorithm (Algorithm 3) is responsible for saving the state of both the parent container and its nested container. It begins by preparing a checkpoint for both containers.

If both containers are ready to be checkpointed, the algorithm commits the checkpoint for each container. If the commit is successful for both containers, the algorithm returns a SUCCESS status. Otherwise, it aborts the checkpoint process for both containers, ensuring that no inconsistent states remain. If either container is not ready for checkpointing, the process is aborted.

The *TransferState* algorithm (Algorithm 4) is responsible for transmitting the checkpointed data of both the parent and nested containers to the destination. It retrieves the checkpoint data for both containers. Once the data is retrieved, it sends the parent and nested container data separately to the destination, marking them with the identifiers "parent" and "nested". This ensures the state data is correctly assigned to each container at the destination.

The *Restore* algorithm (Algorithm 5) is responsible for restoring the state of the parent container and its nested container at the destination. It begins by receiving the parent and nested container data, identified as "parent" and "nested". The algorithm then restores the state of both containers. If the restoration is successful for both containers, the algorithm returns a SUCCESS status; otherwise, it returns a FAILURE status and ensures that no inconsistent state is left behind.

The *Nested Container Manager* module within the *Container Manager* component ensures a reliable and structured migration process, maintaining the integrity and isolation of containers during transitions across cloud and edge environments. This approach enables seamless and efficient migration, ensuring that both parent and nested containers are accurately migrated without data inconsistency.

3.3 SDN-based network management

In FlexiMigrate, the *Network Manager* component implements the *SDN Controller Manager* module, enabling FlexiMigrate to manage and orchestrate network operations for container migration. It achieves this through IP address reassignment paired with DNS updates to ensure service continuity; programmable flow rule management for traffic redirection to migrated containers; adaptive traffic steering to optimize routing during migration events; and dynamic network topology discovery to identify available paths and devices. Taken together, these capabilities guarantee minimal network disruption, reduce network overhead, and maintain application connectivity both during and after migration.

```

1: procedure CHECKPOINT(parentContainer, nestedContainer, destination)
2:   # Prepare a checkpoint for the parent container, capturing its current state.
3:   parentCheckpoint ← PREPARECHECKPOINT(parentContainer)
4:   # Prepare a checkpoint for the nested container, capturing its current state.
5:   nestedCheckpoint ← PREPARECHECKPOINT(nestedContainer)
6:   # Commit the checkpoints or Abort the process.
7:   if (parentCheckpoint.status == READY) and (nestedCheckpoint.status == READY) then
8:     parentCommit ← COMMITCHECKPOINT(parentContainer, parentCheckpoint, destination)
9:     nestedCommit ← COMMITCHECKPOINT(nestedContainer, nestedCheckpoint, destination)
10:    if (parentCommit == SUCCESS) and (nestedCommit == SUCCESS) then
11:      return SUCCESS
12:    else
13:      ABORTCHECKPOINT(parentContainer, parentCheckpoint)
14:      ABORTCHECKPOINT(nestedContainer, nestedCheckpoint)
15:      return FAILURE
16:    end if
17:  else
18:    if parentCheckpoint.status == READY then
19:      ABORTCHECKPOINT(parentContainer, parentCheckpoint)
20:    end if
21:    if nestedCheckpoint.status == READY then
22:      ABORTCHECKPOINT(nestedContainer, nestedCheckpoint)
23:    end if
24:    return FAILURE
25:  end if
26: end procedure

```

Algorithm 3 Checkpoint Algorithm

```

1: procedure TRANSFERSTATE(container, destination)
2:   # Retrieve the checkpoint data for the parent container.
3:   parentData ← GETCHECKPOINTDATA(container)
4:   # Retrieve the checkpoint data for the nested container.
5:   nestedData ← GETCHECKPOINTDATA(nestedContainer)
6:   # Send the parent container's checkpoint data to the destination.
7:   SEND(parentData, destination, "parent")
8:   # Send the nested container's checkpoint data to the destination.
9:   SEND(nestedData, destination, "nested")
10: end procedure

```

Algorithm 4 TransferState Algorithm

```

1: procedure RESTORE(container, destination)
   # Retrieve the parent container's data.
2:   parentData  $\leftarrow$  RECEIVE(destination, "parent")
   # Retrieve the nested container's data.
3:   nestedData  $\leftarrow$  RECEIVE(destination, "nested")
   # Restore the parent container's state.
4:   parentRestore  $\leftarrow$  RESTORECONTAINER(parentData, destination)
   # Restore the nested container's state.
5:   nestedRestore  $\leftarrow$  RESTORECONTAINER(nestedData, destination)
   # Return SUCCESS if both restorations are successful, else FAILURE.
6:   if (parentRestore == SUCCESS) and (nestedRestore == SUCCESS) then
7:     return SUCCESS
8:   else
9:     ABORTRESTORECONTAINER(parentData, destination)
10:    ABORTRESTORECONTAINER(nestedData, destination)
11:    return FAILURE
12:   end if
13: end procedure

```

Algorithm 5 Restore Algorithm

To implement this sophisticated network management, FlexiMigrate employs multiple advanced networking technologies. The OpenFlow protocol [49–51] serves as the primary communication channel between the *SDN Controller Manager* module and network switches, allowing programmatic control of forwarding behavior through the installation of flow rules, which is crucial for rerouting traffic to a container's new IP address after migration. Intent-Based Networking (IBN) [52, 53] allows high-level migration policies to be automatically translated into low-level configurations, enabling the automatic specification of network requirements. VXLAN (Virtual Extensible LAN) technology [54, 55] enables the creation of overlay networks by establishing tunnels between source and destination hosts, decoupling container network identities from physical infrastructure and ensuring continuous connectivity throughout the migration process.

In the *SDN Controller Manager* module, the network management and orchestration process during container migration is handled by the *OrchestrateNetwork* algorithm (Algorithm 6). The algorithm ensures a seamless transition of the container's network connectivity, minimizing disruptions and maintaining active connections. The algorithm begins by retrieving the container's network configuration, which includes VLAN settings, IP subnets, and routing policies. This information is essential to maintaining network consistency at the destination. Next, the SDN switches associated with the source and destination hosts are identified. These switches play a crucial role in updating flow rules to handle traffic redirection. To ensure proper network

integration at the destination, the algorithm assigns a new IP address to the container. This IP address aligns with the destination network's addressing scheme and prevents conflicts. Additionally, the algorithm integrates IBN principles by translating the migration intent into specific network requirements. The migration intent may specify latency constraints, bandwidth reservations, or other network policies. Once the network requirements are defined, the algorithm generates SDN flow rules to manage traffic redirection. These rules ensure that existing connections to the container's original IP address are transparently forwarded to its new IP address at the destination. The source switch is then updated with these flow rules. If this update fails, a rollback mechanism is immediately triggered to restore the original network state. To enable uninterrupted communication, the algorithm establishes a VXLAN tunnel between the source and destination hosts. VXLAN encapsulates the container's network traffic, ensuring seamless data transfer even across different network segments. After the tunnel is set up, flow rules are updated on the destination switch to finalize the migration process. If this step fails, rollback operations are executed on both the source and destination switches to prevent network inconsistencies. Once the migration is complete, outdated flow rules on the source switch are removed to prevent unnecessary redirection and optimize network performance. If all steps execute successfully, the algorithm returns a SUCCESS status, confirming that the migration has been completed with minimal service disruption.

```

1: procedure ORCHESTRATENETWORK(container, sourceHost, destinationHost, migrationIntent)
2:   # Retrieve the container's current network configuration.
3:   networkConfig  $\leftarrow$  GETNETWORKCONFIG(container)
4:   # Identify the SDN switch connected to the source host.
5:   sourceSwitch  $\leftarrow$  GETSWITCH(sourceHost)
6:   # Identify the SDN switch connected to the destination host.
7:   destSwitch  $\leftarrow$  GETSWITCH(destinationHost)
8:   # Obtain a new IP address for the container at the destination.
9:   newIP  $\leftarrow$  GETNEWIP(destinationHost, networkConfig)
10:  # Translate migration intent into specific network requirements using Intent-Based Networking (IBN).
11:  networkRequirements  $\leftarrow$  TRANSLATEINTENT(migrationIntent, networkConfig, sourceHost,
12:    destinationHost)
13:  # Generate SDN flow rules for switches based on the translated network requirements.
14:  sourceFlowRules  $\leftarrow$  GENERATEFLOWRULES(sourceSwitch, networkConfig, destinationHost,
15:    networkRequirements)
16:  destFlowRules  $\leftarrow$  GENERATEFLOWRULES(destSwitch, networkConfig, sourceHost,
17:    networkRequirements)
18:  # Update flow rules on the source switch.
19:  statusSource  $\leftarrow$  UPDATEFLOWRULES(sourceSwitch, sourceFlowRules)
20:  if statusSource == FAILURE then
21:    # If flow rule update fails on the source switch, rollback the changes.
22:    ROLLBACKFLOWRULES(sourceSwitch, sourceFlowRules)
23:    return FAILURE
24:  end if
25:  # Establish a VXLAN tunnel to ensure seamless network connectivity for the migrated container.
26:  CREATEVXLAN(sourceHost, destinationHost, newIP)
27:  # Update the flow rules on the destination switch.
28:  statusDest  $\leftarrow$  UPDATEFLOWRULES(destSwitch, destFlowRules)
29:  if statusDest == FAILURE then
30:    # If flow rule update fails on the destination switch, rollback changes on both switches.
31:    ROLLBACKFLOWRULES(destSwitch, destFlowRules)
32:    ROLLBACKFLOWRULES(sourceSwitch, sourceFlowRules)
33:    return FAILURE
34:  end if
35:  # After successful migration, remove outdated flow rules from the source switch.
36:  DELETEFLOWRULES(sourceSwitch, sourceFlowRules)
37:  return SUCCESS
38: end procedure

```

Algorithm 6 OrchestrateNetwork Algorithm

4 Components and modules in FlexiMigrate

This section provides a detailed description of the modules in each component of the FlexiMigrate framework.

4.1 Migration Manager component

The *Migration Manager* component comprises several key modules that work in concert as follows: *Migration Coordinator*, *Policy Enforcer*, *Resource Allocator*, *Network Orchestrator*, *State Manager*, *Logging & Monitoring*, and *Migration Strategy Selector*.

The *Migration Coordinator* module orchestrates the overall migration process, interfacing with other framework components and ensuring the smooth execution of migration tasks. It uses a Finite State Machine (FSM) to manage the migration lifecycle. Table 2 shows all the states during live container migration along with their definitions, and Fig. 4 illustrates how the state machine changes during live container migration.

As shown in Fig. 4, this process starts with a PENDING state (1N), where the migration request is received and a set of initialization tasks need to be performed. Upon successful initialization, it transitions to the PLANNING state (2N), invoking the *Decision Engine* component to determine

Table 2 Machine States with their Description

Machine State	Description
PENDING	The initial state where the migration request is received but not yet initiated.
PLANNING	The state where the migration strategy is determined, including resource allocation and path selection.
PREPARATION	The state where the necessary containers are prepared, and necessary configurations are set up for migration.
EXECUTION	The active state of migrating the container, where the actual transfer of data and state occurs.
VERIFICATION	The state where the migration is confirmed, ensuring that the container is functioning correctly in the new environment before finalizing the process.
COMPLETED	The final state indicating that the migration process has successfully concluded, with all tasks verified and the container fully operational in its new location.
FAILED	The state indicating that the migration process has encountered an error, preventing successful completion and requiring intervention for resolution.
ROLLBACK	The state where the system attempts to revert the migration to the original state after a failure or critical error during the migration process.

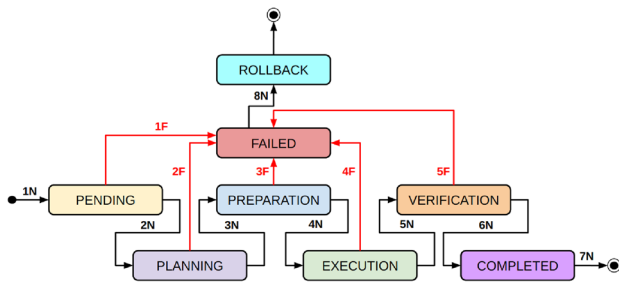


Fig. 4 Interaction between Different States in Container Migration

the optimal migration strategy, allocating resources at the destination via the *Resource Allocator* component, and generating a detailed migration plan. It then passes to the PREPARATION state (3N), where the *Container Manager* component prepares containers for migration, the *Network Manager* component sets up the necessary network configurations, and the checkpointing mechanisms are initialized with the *State Synchronizer* component. Once preparations are complete (4N), the EXECUTION state is started, triggering the actual migration process, monitoring progress, handling runtime issues, and coordinating state transfer between the source and destination. Following execution, the workflow moves to the VERIFICATION state (5N), performing post-migration checks on container functionality, verifying network connectivity and application responsiveness. If all verifications pass (6N), the workflow enters the COMPLETED state, finalizing resource allocations and network configurations, updating system metadata and logs, and notifying relevant components and users of the successful migration. However, if at any point during the container migration process a critical error occurs, the workflow transitions to the FAILED state (1F, 2F, 3F, 4F, and 5F), where error handling and recovery procedures are initiated, detailed error information is logged for analysis, and rollback or cleanup operations are attempted as necessary (8N). This structured workflow, using a state machine approach, ensures a systematic, controlled, and error-resistant mechanism for container migration, with each state building upon the successes of the previous ones and providing clear pathways for both successful completion and error management⁷.

In the *Migration Manager* component, the *Policy Enforcer* module enforces the migration policies specified in FlexiMigrate, ensuring compliance with predefined policies during the container migration process. It utilizes a rule-based policy specification model to define migration policies. Each policy is in the form of a rule, defined as follows:

```
POLICY <policy_name>:
  CONTEXT:
    <context_variables>
  CONDITIONS:
    <condition_expressions>
  ACTIONS:
    <action_specifications>
  CONSTRAINTS:
    <constraint_definitions>
  PRIORITY: <priority_level>
```

where:

- <context_variables> define the dynamic environmental factors to be considered.
- <condition_expressions> are complex boolean expressions that can include nested logical operations, temporal conditions, and probabilistic thresholds, and must be satisfied.
- <action_specifications> detail the operations to be performed, including compound actions and cascading effects.
- <constraint_definitions> specify limitations on resources, timing, or other factors for container migration.
- <priority_level> determines the policy's precedence in case of conflicts.

The *Policy Enforcer* module evaluates and enforces multifaceted policies for container migration in real-time, significantly enhancing the flexibility and adaptability of the migration process. By defining and enforcing rule-based policies, FlexiMigrate can effectively handle a wide range of scenarios, enabling context-aware decision-making, complex condition evaluation, and sophisticated action chaining during the migration process. This module allows the system to respond to both immediate and evolving conditions while maintaining compliance with predefined policies. Example 1 illustrates the sophistication of a rule-based migration policy in the FlexiMigrate framework.

⁷ See different log scenarios demonstrating state machine transitions logged by the FlexiMigrate framework at: <https://github.com/ahmadpanah/FlexiMigrate-Framework>.

Example 1 A policy can be defined as follows:

Example 1. *A policy can be defined as follows:*

```
POLICY adaptive_load_balancing:
CONTEXT:
  time_of_day, network_congestion, service_type
CONDITIONS:
  (Source_cpu_utilization > 80% AND destination_cpu_utilization < 50%)
  OR
  (time_of_day IN peak_hours AND service_type == 'critical')
  OR
  (PROBABILITY(network_congestion) < 0.2)
ACTIONS:
  allow_migration AND
  set_priority(high) AND
  trigger_load_balancer_reconfiguration()
CONSTRAINTS:
  max_concurrent_migrations <= 5
  migration_duration < 300s
PRIORITY: 2
```

The policy illustrated in Example 1 allows container migration when any of the specified conditions are satisfied. Migration is triggered for overload mitigation when the source CPU utilization is high (Source_cpu_utilization > 80%) and the destination has sufficient available capacity (destination_cpu_utilization < 50%). It is also activated to prioritize critical services during peak hours (time_of_day IN peak_hours AND service_type == 'critical'), regardless of CPU load. Furthermore, migration is initiated proactively when network congestion is low (PROBABILITY(network_congestion) < 0.2), ensuring optimal performance. The use of the OR operator guarantees that migration is executed based on distinct optimization objectives. For example, even if CPU utilization is balanced, the policy allows preemptive migration to leverage idle network bandwidth when congestion is minimal. To maintain system stability and prevent excessive migrations, constraints such as max_concurrent_migrations and migration_duration are enforced.

In the FlexiMigrate framework, migration policies are defined by system administrators or DevOps teams through declarative YAML files or REST APIs, adhering to infrastructure-as-code principles. During policy enforcement, the *Policy Enforcer* module resolves conflicts by prioritizing rules with higher priority values (e.g., Priority 1 overrides Priority 2). If multiple policies have the same priority, the system alerts administrators or DevOps teams to address the conflict, ensuring a controlled and predictable migration process.

The *Resource Allocator* module operates in close collaboration with the *Resource Monitor* component to guarantee optimal resource allocation at the destination host. By leveraging a combination of static analysis and dynamic monitoring, this module assesses resource availability and workload demands in real time. This dual approach allows

for a more precise resource allocation strategy, ensuring that resources are efficiently utilized to meet the requirements of the migrating containers.

The *Network Orchestrator* module collaborates with the *Network Manager* component within the FlexiMigrate framework to handle network reconfigurations necessary for maintaining connectivity during and after container migration using SDN principles [56–59]. This approach encompasses a multi-step process, as explained in Sect. 3.3, to manage and orchestrate network operations during container migration. By leveraging the SDN-based approach, FlexiMigrate achieves unprecedented flexibility in network management, enabling rapid adaptation to evolving migration requirements. This flexibility is achieved due to the decoupling of the control plane from the data plane and the capability for centralized programmability and real-time traffic engineering in network management [60, 61]. Unlike traditional static network management, SDN can dynamically reconfigure flow rules and routing paths based on real-time demands (e.g., live migration demands), ensuring seamless connectivity. The centralized SDN controller provides a global view of the network, enabling rapid adaptation to changing migration requirements through fine-grained control over traffic flows and resource allocation. While SDN is not strictly necessary for container migration, its integration significantly enhances adaptability and efficiency, surpassing the limitations of traditional network management approaches.

The *State Manager* module collaborates closely with the *State Synchronizer* component to effectively manage the checkpoints, enabling smooth transformation and restoration of container states during the migration process. It employs a Delta-Based approach [62], which focuses on capturing and transferring only the changes made to the

container state since the last checkpoint. This technique significantly reduces the amount of data that needs to be transferred, optimizing network bandwidth usage and accelerating the migration process [14]. By minimizing data transfer, the Delta-Based approach enhances overall system performance and ensures a more efficient restoration of the container's operational state at the destination. This technique not only speeds up migration but also helps maintain consistency and reduce downtime during the container migration process.

The *Logging & Monitoring* module in the FlexiMigrate framework plays a crucial role in ensuring visibility and accountability during the container migration process. This module tracks and records all relevant events and actions within the framework, such as user requests, migration decisions, resource allocation, network configurations, and state transfers. By collecting and analyzing these logs, the FlexiMigrate framework can monitor system health and identify potential issues during migration. This module, as a logging and monitoring module, is implemented based on the ELK stack (Elasticsearch, Logstash, Kibana)⁸, extended with Prometheus⁹. Structured JSON logs produced by all components are picked up by distributed Logstash agents and stored in a centralized Elasticsearch database for efficient indexing and querying. Prometheus fetches real-time metrics, representing key performance indicators and overall resource utilization. For visualization, the module includes customized Kibana dashboards¹⁰ for log analytics and Grafana interfaces¹¹ for real-time metric display. This module provides comprehensive visibility of migration processes, rapid troubleshooting, and continuous insights for improving the framework's performance in container migration.

The *Migration Strategy Selector* module is responsible for determining the most appropriate migration strategy to employ for a given container migration request. Table 3 shows different migration strategies that can be selected by this module: Cold Migration, Pre-Copy Migration, Post-Copy Migration, or Live Migration. Cold Migration [63] is typically selected when service downtime is acceptable, or when the container is not time-critical. Pre-Copy [64, 65] Migration begins the transfer process before the container stops running, ensuring lower downtime but requiring higher network bandwidth for multiple rounds of copying. Post-Copy Migration [66, 67] transfers the container's state after stopping it, resulting in shorter migration time but with potential risks in case of failure. Live Migration [63]

Table 3 Migration Strategies for Container Migration

Migration Strategy	Description
Cold Migration	It involves stopping the container, transferring its image, and then restarting it on the target host. This strategy incurs the highest downtime.
Pre-Copy Migration	It begins copying the container's memory while it is still running, followed by a final copy when the container is stopped. This reduces downtime but can increase network overhead.
Post-Copy Migration	It starts transferring the container after halting it, moving memory and state after the container is stopped. This minimizes the overall migration time but may increase risk if any errors occur during the process.
Live Migration	It periodically saves the container's state and data, allowing for restoration on the target host. This strategy effectively balances downtime and resource usage.

is chosen in scenarios where both minimal downtime and resource efficiency are critical, such as for real-time applications or sensitive services.

The *Migration Strategy Selector* module utilizes the Analytic Hierarchy Process (AHP) [68–70] as a multi-criteria decision-making algorithm to systematically evaluate various performance metrics, such as memory usage, CPU usage, network bandwidth, disk I/O, container size, and downtime tolerance—crucial factors in determining the optimal migration strategy. It enables the decision problem to be decomposed into a hierarchy of criteria (i.e., memory usage, CPU usage, and network bandwidth) and sub-criteria (disk I/O, container size, and downtime tolerance), which are evaluated by assigning weights relative to their importance. These weights are synthesized to obtain an overall score for each migration strategy. This multi-criteria decision algorithm ensures that the chosen strategy is the best, considering the various performance metrics affected by live container migration.

4.2 Resource monitor component

The *Resource Monitor* component is tasked with the continuous collection and analysis of performance metrics and resource utilization data across heterogeneous cloud and edge computing environments. This component provides real-time insights for the FlexiMigrate framework, thereby enhancing decision-making related to container migration. The component comprises two key modules: the *Performance Metrics Collector* module and the *Resource Utilization Analyzer* module.

The *Performance Metrics Collector* module gathers a comprehensive range of performance-related data from containers, hosts, and network infrastructure. Table 4 provides a list of performance metrics collected by this module with their descriptions. While these performance metrics are

⁸ <https://www.elastic.co/elastic-stack>.

⁹ <https://prometheus.io/>.

¹⁰ <https://www.elastic.co/kibana>.

¹¹ <https://grafana.com/>.

Table 4 Performance Metrics Collected by the Performance Metrics Collector Module with Their Definitions and Thresholds

Performance Metric	Definition	Default Threshold
CPU Usage	Percentage of CPU time used by the container or host.	< 80%
Memory Usage	Amount of RAM currently being used by the container or host.	< 70%
Disk I/O	Number of read and write operations per second on storage devices.	< 100 ops/sec
Network Throughput	Amount of data transferred over the network per unit of time.	< 500 MB/min
Container Startup Time	Time taken for a container to become operational.	< 5 sec
Response Time	Time taken to respond to a request (applicable for web services).	< 200 ms
Error Rate	Frequency of errors occurring in the container or application.	< 1%
Network Latency	Time taken for data to transfer across the network.	< 100 ms
CPU Load Average	Average system load over 1, 5, and 15-minute intervals.	< 2.0
Memory Page Faults	Number of page faults occurring in the system.	< 50 faults/min
Network Packet Loss	Percentage of packets lost during network transmission.	< 1%
Disk Queue Length	Number of I/O requests waiting to be processed.	< 5 requests
Container Restart Count	Number of times a container has been restarted.	< 3 restarts in 1 hour
Network Connection Count	Number of active network connections.	< 100 connections
System Call Rate	Frequency of system calls made by the container or host.	< 1000 calls/sec

initially configured with default threshold values, system administrators or DevOps teams can adjust them based on workload characteristics, application requirements, or predefined SLAs. This flexibility ensures that container migrations dynamically adapt to real-time operational demands and infrastructure conditions, optimizing resource utilization and overall system performance.

The *Performance Metrics Collector* module utilizes various techniques for data acquisition, including system calls for low-level metrics collection, integration with container runtime APIs such as Docker for managing containerized applications, kernel-level probes like `psutil` for detailed system performance data, and network protocol analyzers such as `net_io` for capturing traffic patterns. Table 5 presents a list of performance metrics along with the tools and commands used to measure them.

To efficiently collect performance metrics and monitor resource utilization in edge computing environments, the *Resource Monitor* component employs a hybrid push-pull model optimized for edge deployments. Each edge node runs a lightweight agent that aggregates performance metrics

Table 5 Collected Performance Metrics and Their Measurement Tools

Performance Metric	Tools/Commands
CPU Usage	<code>psutil.cpu_percent()</code> (Python library), <code>top</code> , <code>htop</code>
Memory Usage	<code>psutil.virtual_memory()</code> , <code>free -m</code> , Docker API/system/stats endpoint
Disk I/O	<code>iostat</code> , <code>docker stats</code> , <code>fio</code> (synthetic workloads)
Network Throughput	<code>iftop</code> , <code>nload</code> , <code>tcpttrack</code> , Docker Network API
Container Startup Time	<code>time docker run....</code> , custom shell scripts with timestamps
Response Time	<code>curl -w"@curlformat"-o/dev/null</code> , Prometheus Exporter
Error Rate	Prometheus Alertmanager, custom logs parsed with <code>grep/awk</code>
Network Latency	<code>ping</code> , <code>traceroute</code> , <code>iperf3</code>
CPU Load Average	<code>uptime</code> , <code>sar -q</code> (sysstat package)
Memory Page Faults	<code>vmstat</code> , <code>/proc/vmstat</code>
Network Packet Loss	<code>ping</code> , <code>nping</code> (Nmap), Prometheus network monitoring
Disk Queue Length	<code>iostat -x</code> , <code>sar -d</code>
Container Restart Count	Docker API/ <code>containers/json?all=1</code> , <code>journalctl</code> (systemd logs)
Network Connection Count	<code>netstat -an grep ESTABLISHED wc -l</code> , <code>ss -s</code>
System Call Rate	<code>strace -c</code> , <code>perf stat</code>

using minimal system calls and Docker API queries. Metrics are batched every 500ms to reduce network overhead and compressed with `zstd` for bandwidth efficiency. During network instability, collected metrics are temporarily stored in a circular buffer (up to 1MB per node) and transmitted once connectivity is restored. A centralized Prometheus instance scrapes critical performance metrics (e.g., container health) every 2 seconds via HTTP/2, while less frequent data (e.g., disk I/O) is pushed asynchronously. Additionally, adaptive sampling dynamically adjusts the collection frequency based on system load: when CPU usage exceeds 70%, the sampling interval increases to 1 second to reduce monitoring overhead, whereas under low load conditions ($\leq 30\%$), the interval extends to 5 seconds to conserve resources. This adaptive approach balances accuracy with efficiency, as validated in Sect. 5 under throttled bandwidth conditions (50–300 Mbps).

In the *Resource Monitor* component, the *Resource Utilization Analyzer* module processes the raw data collected by the *Performance Metrics Collector* module to extract meaningful insights and identify patterns that may warrant container migration or resource reallocation. The module employs a threshold-based mechanism [71, 72] to analyze resource utilization. When these metrics exceed the specified thresholds, it flags potential issues and informs the *Decision Engine* component to consider predefined policies

for potential container migration. The utilization of a threshold-based mechanism in the *Resource Utilization Analyzer* module ensures that resource bottlenecks and inefficiencies are promptly identified, thereby facilitating timely decision-making and optimizing resource allocation within the cloud and edge computing environments.

4.3 Decision Engine component

The *Decision Engine* component is essential for analyzing workloads, optimizing resource allocation, and planning migrations. It leverages advanced machine learning techniques to enhance the overall system's performance and efficiency during container migration. The core of its functionality relies on the Random Forest algorithm [73], a robust ensemble learning method that builds multiple decision trees to improve prediction accuracy and resilience against overfitting.

The Random Forest algorithm works by constructing a collection of decision trees using a technique known as bootstrapping, which samples data with replacement. Each tree in the forest is trained on a random subset of the data, making independent predictions. The final prediction is then derived by aggregating the results from all trees. For classification tasks, majority voting is used to determine the output, while for regression tasks, the average of the outputs from each tree is used.

Mathematically, the Random Forest algorithm can be broken down into the following steps:

1. *Bootstrap Aggregating (Bagging)*: A random subset of the training data, $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, is selected by sampling with replacement to create multiple training sets, $D_1, D_2, \dots, D_T$, where T is the number of trees in the forest.
2. *Tree Construction*: For each subset D_t , a decision tree T_t is constructed. The tree is grown by recursively splitting the data using a feature that provides the best split at each node based on a criterion (e.g., Gini impurity for classification or mean squared error for regression). This process continues until a stopping criterion is met (e.g., maximum depth or minimum number of samples per leaf).
3. *Prediction*: Once the trees are built, they make predictions based on the input features x . For a classification task, the output is the majority class predicted by all the trees:

$$\hat{y} = \text{majority vote}(T_1(x), T_2(x), \dots, T_T(x))$$

For regression, the output is the average of the predictions from all trees:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T T_t(x)$$

4. *Feature Importance*: Random Forest also computes the importance of each feature based on how much they contribute to reducing the error in the model. Features that consistently improve the split quality in the decision trees are considered more important.

The Random Forest algorithm is applied across three key modules within the *Decision Engine* component: the *Workload Analyzer* module, the *Resource Optimizer* module, and the *Migration Planner* module. These modules use the machine learning model to process historical data, make predictions, and optimize decisions that guide the migration process. The algorithm works by training on random subsets of data, enabling each decision tree to independently make predictions. The final output is derived by aggregating these predictions.

The *Workload Analyzer* module uses the Random Forest algorithm to analyze past workload data, categorize current workload patterns, and forecast future resource demands. It processes high-dimensional data, capturing complex relationships between factors such as CPU usage, memory consumption, network traffic, and temporal workload variations (e.g., hourly, daily, weekly). By aggregating the predictions of individual trees, the *Workload Analyzer* module provides robust and reliable estimations of the system's resource requirements under various scenarios.

In parallel, the *Resource Optimizer* module applies the Random Forest model to inform decisions regarding resource allocation. Trained on performance and allocation data, it maps system states to optimal resource configurations. The algorithm's feature importance ranking allows it to identify critical factors such as CPU utilization, memory use, storage demands, and network performance, which influence the optimal allocation of resources. This enables the *Resource Optimizer* module to adjust resources dynamically across heterogeneous cloud and edge environments, effectively balancing performance needs with cost considerations.

The *Migration Planner* also incorporates the Random Forest algorithm to evaluate different migration strategies. It is trained on historical migration data that includes CPU cost, memory cost, downtime, and transfer costs. By capturing the complex relationships between these factors, the model enables the *Migration Planner* module to evaluate multiple migration scenarios and select the one that minimizes downtime and resource usage while maximizing efficiency.

The synergy between the three modules is key to the effectiveness of the *Decision Engine* component. The Random Forest algorithm's ability to handle non-linear relationships, reduce overfitting, and provide interpretable feature

importance allows the system to make data-driven decisions that are both accurate and adaptable. As the system continuously learns from new data, it improves its predictions and decisions, ensuring that container migrations are always optimized for the given conditions of the cloud and edge environments.

4.4 Container Manager component

The *Container Manager* component comprises three primary modules: the *Runtime Controller*, *Nested Container Manager*, and *Image Manager*, which manage container lifecycle operations, facilitate seamless migrations, and ensure efficient image management across heterogeneous cloud and edge environments.

The *Runtime Controller* module acts as an abstraction layer between the FlexiMigrate framework and various container runtimes, enabling seamless operation across different container technologies and orchestration platforms. By implementing a unified interface for container operations, the module translates FlexiMigrate's standardized commands into runtime-specific API calls, using design pattern such as strategy pattern [74]. It likely employs the Container Runtime Interface (CRI)¹² as a communication protocol, alongside gRPC¹³ for efficient, language-agnostic interactions, and Protocol Buffers¹⁴ for cross-platform data serialization. The module ensures runtime-agnostic container lifecycle management, leveraging tools such as CRIU¹⁵ for checkpointing and restoration. Additionally, it maintains an extensible architecture to support new container runtimes, ensuring a consistent interface for the entire framework.

The *Nested Container Manager* module is central to FlexiMigrate's innovative approach to live container migration. By utilizing a nested container architecture as explain in Sect. 3.2, it ensures seamless migration while preserving the integrity and isolation of application containers. This module coordinates the migration process through a dynamic parent container, which manages resources and abstracts migration complexities from the application layer. It leverages Linux kernel features, such as namespaces and cgroups, to implement resource isolation and hierarchical container structures. The module facilitates migration using CRIU for checkpointing and a two-phase commit (2PC) protocol [75–77] to ensure state consistency across the entire container stack. It integrates with container orchestration platforms, supporting automated migration while maintaining compatibility with existing infrastructures.

The *Image Manager* module in the *Container Manager* component orchestrates the efficient handling and distribution of container images across heterogeneous cloud and edge environments. At its core, this module employs advanced techniques to optimize image management, including delta-based version updates [78] and content-addressable storage for layer deduplication [79]. A *layer* in the context of container images refers to a set of files and metadata that form a part of the overall image. Each layer represents a set of changes made to the image, such as the installation of software packages or modifications to configuration files. Container images are typically built by stacking layers on top of each other, with each layer representing a specific set of instructions. This layered structure enables efficient image storage, sharing, and versioning, as only the changes (layers) need to be transmitted or stored, rather than the entire image. Delta-based version updates implement a binary diff algorithm to compare image versions and generate compact delta files. When updating, only these delta files are transferred and applied to the existing image, minimizing the amount of data transferred during container image updates. This reduces network usage and speeds up the update process, which is particularly beneficial in cloud and edge computing environments. Content-addressable storage for deduplication of image layers is implemented through a storage backend that employs a rolling checksum algorithm [80], such as Rabin fingerprinting [81], to identify and track unique data blocks within image layers. This method generates a series of checksums for overlapping blocks of data, allowing for efficient identification of duplicate content, even when it appears at different offsets within files. The generated checksums serve as unique identifiers for storing and retrieving layers, eliminating redundant storage of identical data across different images, saving significant storage space, and facilitating deduplication across the entire system. Furthermore, the *Image Manager* module integrates with popular container registries, such as Docker Hub¹⁶ and Quay.io¹⁷, by implementing APIs and protocols to interact with various registries. This ensures broad compatibility and allows for efficient image pulling from various sources, enhancing the framework's flexibility and enabling seamless operation across diverse container ecosystems. Through this comprehensive suite of features, the *Image Manager* module establishes itself as an indispensable tool for streamlining container image logistics, ultimately contributing to FlexiMigrate's goal of enabling seamless and efficient live container migration in complex, distributed cloud and edge computing environments.

¹² <https://kubernetes.io/docs/concepts/architecture/cri/>.

¹³ <https://grpc.io/>.

¹⁴ <https://protobuf.dev/>.

¹⁵ <https://criu.org/>.

¹⁶ <https://hub.docker.com/>.

¹⁷ <https://quay.io/>.

4.5 Network Manager component

The *Network Manager* component in the FlexiMigrate framework oversees the orchestration of network-related operations to ensure seamless container migration and uninterrupted service availability. It leverages SDN principles [56–59] along with advanced network management techniques to address the complex networking challenges associated with live container migration in cloud and edge environments, as explained in Sect. 3.3. The *Network Manager* comprises three primary modules: the *SDN Controller Connector*, the *DNS Controller Manager*, and the *Network Traffic Manager*.

The *SDN Controller Connector* module serves as a bridge between FlexiMigrate and various SDN controllers, utilizing standardized interfaces such as the OpenFlow protocol and RESTful APIs. Through these interfaces, it programmatically manages and reconfigures network infrastructures across heterogeneous cloud and edge environments. The *SDN Controller Connector* module is implemented to interface with the OS-ken SDN controller, version 2.11.2, leveraging its Python-based REST API and OpenFlow protocol support. OS-ken was selected for its comprehensive feature set, including robust support for OpenFlow 1.3, its modular architecture facilitating custom application development, and its active community providing extensive documentation and support. The *SDN Controller Connector* module implements key OS-ken API interactions, including retrieving network topology information via the GET /stats/switches and GET /topology/links endpoints, dynamically adding and removing flow rules using the POST /stats/flowentry/add and POST /stats/flowentry/delete endpoints, respectively, and monitoring switch status via the GET /stats/desc/dpid endpoint. These operations enable the FlexiMigrate framework to programmatically manage network traffic flow, ensure continuous network connectivity during container migration, and dynamically adapt to changing migration requirements within the software-defined network. Dynamic flow rule management is executed using a 2PC protocol [75–77], which ensures atomic updates across distributed switches. This process involves preparing flow rule modifications on all affected switches, followed by a commit phase that simultaneously activates the changes, thereby maintaining network consistency during container migration. To handle network policy translation and enforcement, the module converts high-level migration policies into low-level network configurations. This translates FlexiMigrate’s internal network operations into protocol-specific instructions, ensuring seamless integration with diverse network infrastructures and enabling efficient container migration across multi-cloud and edge computing environments.

In the *Network Manager* component, the *DNS Controller Manager* module ensures consistent and up-to-date DNS records as containers migrate between hosts. This functionality guarantees that service discovery mechanisms remain operational, allowing clients to locate migrated containers with minimal disruption. The module offers advanced capabilities, including atomic DNS record updates to prevent inconsistent states, intelligent time to live (TTL) management to balance update propagation and caching efficiency, support for various DNS protocols and server implementations, and integration with service mesh and service discovery systems.

The *Network Traffic Manager* module within the *Network Manager* component provides intelligent traffic steering mechanisms to minimize packet loss and latency throughout the migration process. It ensures continuous network connectivity during and after container migration. Key features include seamless traffic handover using programmable data planes, stateful flow migration to maintain session continuity, and adaptive tunnel management for optimized network performance. The module employs the Long Short-Term Memory (LSTM) neural network [82] to predict traffic patterns and optimize network paths. It implements a dynamic routing algorithm based on the multi-commodity flow problem [83], considering factors such as link capacity, network latency, and predicted traffic loads to compute optimal paths for migrating containers. The module also manages DNS record consistency during migrations using a distributed 2PC protocol. This process involves creating temporary DNS records with new IP addresses, followed by an atomic switch to the new records upon successful preparation across all DNS servers. To handle concurrent migration events, the module implements a conflict resolution mechanism based on the Raft consensus algorithm [84–86] to ensure data integrity during the migration process.

4.6 State Synchronizer component

The *State Synchronizer* component is entrusted with ensuring consistent and efficient state transfer during container migration. The *State Synchronizer* component comprises three primary modules: the *Checkpoint Controller*, *Delta Tracker*, and *State Restoration*.

The *Checkpoint Controller* module is tasked for capturing a container’s complete state at a specific point in time. It generates consistent snapshots of the container’s memory, file system, and network connections by temporarily suspending execution to maintain state consistency. The module systematically captures memory pages, file system state, and active network connections. To optimize efficiency, it uses a Copy-on-Write mechanism [87, 88] for incremental checkpointing, memory page deduplication [89, 90] to eliminate redundant data, and compression [91, 92] to reduce checkpoint size. The checkpoint, along with relevant metadata, is stored in

a distributed system, where its integrity is verified before the container's execution resumes. This approach minimizes downtime and resource consumption while ensuring data consistency. By integrating with container runtime-specific APIs and distributed storage, the module achieves high compatibility and scalability across cloud and edge environments.

In the *State Synchronizer* component, the *Delta Tracker* module continuously monitors incremental changes to the container state between checkpoints. As a result, during container migration, only these delta changes are transferred, significantly reducing the amount of data that needs to be moved. This module enables efficient incremental state transfer and reduces the overall data volume during migration. It provides sophisticated capabilities, including real-time tracking of memory page modifications, file system change monitoring using inotify or similar mechanisms, network state delta computation, and adaptive triggering of incremental checkpoints based on delta volume or time thresholds.

The *State Restoration* module is charged with reconstructing the container state on the target host during migration by applying checkpoints and deltas to accurately recreate the execution environment. It also includes resilient restoration features, such as error detection, recovery mechanisms, and post-restoration validation checks. These capabilities are implemented using a technique known as *adaptive buffering* [93], which addresses the multifaceted challenges inherent in live migration, particularly in heterogeneous edge and cloud computing environments.

5 Performance evaluation

In this section, we first explain our experimental setup, followed by a detailed implementation of container migration, and then present and analyze the experimental results.

5.1 Experimental setup

The experimental setup was designed to simulate a realistic edge computing environment using virtualization, as shown in Fig. 5, with hardware and software specifications detailed in Table 6. Built on VMware vSphere, the infrastructure utilized VMware vCenter Server 7.0 Update 3g for centralized management. Two VMware ESXi hosts, each equipped with dual Intel Xeon Gold 6248R processors (96 cores, 192 threads), 284 GB of DDR4 RAM, and 2 TB of SSD storage, provided ample resources for virtualized workloads. Network connectivity was managed by dual 1 Gbps NICs on each host, supported by a MikroTik CCR1036-12G-4S switch running RouterOS v7.8.

To simulate edge nodes, 20 virtual machines were configured with specifications resembling low-resource devices such as Raspberry Pi: a quad-core ARM Cortex-A72 processor, 8 GB of RAM, and 128 GB of virtual SSD storage. Network constraints, including throttled migration traffic (50–300 Mbps) and latency injections (20–150 ms), were applied to simulate typical edge-to-cloud communication patterns and resource variability using tc netem tools.

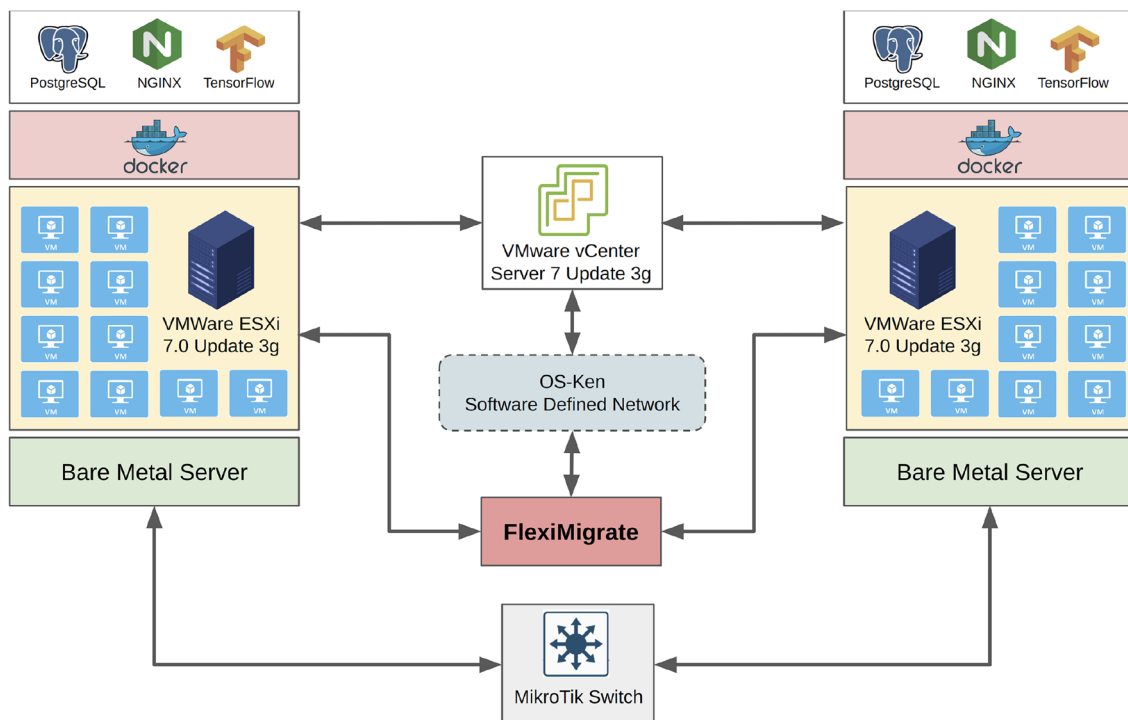


Fig. 5 Virtual Infrastructure in Our Experiments

Table 6 Hardware and Software Specifications for Experimental Setup

Component Category	Specific Component	Version/Specifications
Virtualization	VMware vSphere	7.0 Update 3g
	VMware ESXi	7.0 Update 3g
	VMware vCenter Server	7.0 Update 3g
Hardware (Hosts)	CPU (ESXi hosts)	Dual Intel Xeon Gold 6248R (96 cores, 192 threads)
	RAM (ESXi hosts)	284 GB DDR4
	Storage (ESXi hosts)	2 TB SSD
	Network Interface Cards (NICs)	Dual 1 Gbps
	Network Switch	MikroTik CCR1036-12G-4S
Network Infrastructure	Network OS	RouterOS v7.8
	Processor	Quad-core ARM Cortex-A72
VM Specifications	RAM	8 GB
	Storage	128 GB virtual SSD
	Docker Engine	v27.3.1
Software	Python Language	3.12.6
	NGINX Web Server	1.26.2
	PostgreSQL Database	16.4
	TensorFlow Lite	2.x
	Docker SDK for Python	4.0.0+
Libraries/Tools	NetworkX	2.0+
	os-ken	2.0+
	Elasticsearch	8.14.3
	Prometheus Client	0.7.0+
	bsdiff4	1.1.5+
	gRPC	1.30.0+

Background workloads, including lightweight machine learning inference and IoT data aggregation, were continuously run on the edge nodes to simulate dynamic resource usage and multi-tasking scenarios. This setup allowed us to assess FlexiMigrate's performance under typical distributed edge infrastructure conditions, such as resource contention, fluctuating latency, and bandwidth constraints.

The software stack was built on VMware vSphere 7.0 Update 3g as the virtualization platform, with Docker Engine v27.3.1 installed on each virtual machine to handle containerized applications. The experiment included three distinct containerized workloads: a web server (NGINX 1.26.2) serving static content, a PostgreSQL 16.4 database for data persistence, and a machine learning inference service using TensorFlow 2.x Lite for executing pre-trained models.

The FlexiMigrate framework itself was built upon a comprehensive set of libraries and tools. Python 3.12.6 served as the primary programming language, chosen for its versatility and extensive ecosystem. Key components included the Docker SDK for Python (docker-py) 4.0.0+ for container management, NetworkX 2.0+ for graph-based network analysis, and os-ken 2.0+ as the SDN controller. For monitoring

and logging, the framework integrated Elasticsearch 8.14.3 and Prometheus Client 0.7.0+. Data processing and transfer were facilitated by bsdiff4 1.1.5+, while gRPC 1.30.0+ was employed for efficient inter-service communication. TensorFlow 2.x rounded out the stack, providing machine learning capabilities crucial for adaptive decision-making within the migration processes. The code for the FlexiMigrate framework is available online.¹⁸

To calibrate the threshold values for performance metrics shown in Table 4, thresholds were derived iteratively through empirical analysis of over 50 containerized workloads, including web servers, databases, and ML inference tasks. For instance, the CPU Usage threshold of 80% was determined because exceeding this value led to an exponential increase in latency due to scheduler contention, which was measured using perf stat. The Memory Usage threshold was set at 70%, providing a 30% buffer for sudden spikes without triggering swap, as validated through vmstat. For Disk I/O, a threshold of 100 operations per second (ops/sec) was established for SATA SSDs, with NVMe thresholds automatically scaled to 500 ops/sec based on device detection. These threshold values were further refined using logistic regression on historical outage data to minimize false positives.

We evaluated the performance of the FlexiMigrate framework in comparison to the ProKube framework [19]¹⁹, the LIMOCE framework [34]²⁰, and a CRIU-based migration framework²¹ across three distinct scenarios:

- **Single Container Migration:** In this scenario, we migrated a single container running the NGINX web server from one edge node to another, assessing the efficiency of the migration process.
- **Multi-Container Application Migration:** This scenario involved migrating a multi-container application comprising 200 containers, including 100 NGINX web servers and 100 PostgreSQL databases. These containers were migrated across different edge nodes to evaluate FlexiMigrate's ability to manage interdependent services. This specific migration scenario focused on the NGINX web server-PostgreSQL database pair, demonstrating coordinated state transfer and network reconfiguration for dependent services.
- **Migration under Network Congestion:** Here, we introduced network congestion, simulating 50% packet loss during the migration of a TensorFlow Lite container, aiming to assess the resilience and performance of FlexiMigrate under adverse network conditions.

¹⁸ <https://github.com/ahmadpanah/FlexiMigrate-Framework>.

¹⁹ <https://github.com/babarAli93/Prokuba>.

²⁰ <https://github.com/ahmadpanah/LIMOCE>.

²¹ <https://github.com/blead/cloudhopper>.

To evaluate the performance of the FlexiMigrate framework in these scenarios, we utilized the following performance metrics:

- **Total Migration Time:** The total duration required to complete the migration of a container from the source node to the destination node.
- **Downtime During Migration:** The total period during which the container is unavailable for use due to the migration process.
- **Network Overhead:** The total volume of data transmitted over the network during the migration process, including both the container data and any additional meta-data or control information.
- **Average CPU Usage:** The average CPU usage on both the source and destination nodes throughout the migration process.
- **Peak Memory Usage:** The maximum memory consumption recorded on the source and destination nodes at any point during the migration.

To further explore the novel contribution of FlexiMigrate's nested container architecture, we conducted two targeted evaluations:

- **Heterogenous Platform Migration Verification:** We initially attempted to migrate a basic stateful web application container from a Kubernetes (K8s) node to a Docker Swarm node. The same container was then encapsulated within FlexiMigrate's nested container architecture and migrated between the same Kubernetes (K8s) and Swarm nodes using the FlexiMigrate framework. This experiment aimed to demonstrate the necessity and capability of the nested container architecture in overcoming orchestration platform dependency issues and enabling orchestrator-agnostic migration in heterogeneous environments.
- **Quantification of Runtime Performance Overhead:** This experiment was designed to assess the steady-state impact of the nested container architecture in the FlexiMigrate framework on system performance using Kubernetes (K8s). We measured key performance metrics for the NGINX web server, PostgreSQL database, and TensorFlow Lite workloads, both in a standard container

setup and within FlexiMigrate's nested container architecture, on the same edge node, without performing any migration. The objective was to quantify the resource consumption and performance overhead induced by the nested container architecture during normal runtime operation.

We used the same performance metrics that were used for evaluating the FlexiMigrate framework in comparison with existing migration frameworks to evaluate the performance of the nested container architecture of FlexiMigrate in the verification of heterogeneous platform migration.

For the quantification of runtime performance overhead, the following performance metrics were utilized to compare the nested container architecture in FlexiMigrate versus standard deployment:

- **Average CPU Usage:** The average CPU resources consumed by the workload container(s) and associated framework processes during steady-state operation.
- **Average Memory Usage:** The average memory usage of the workload container(s) and framework processes during steady-state operation.
- **Disk I/O Throughput:** The read/write rate for disk-bound loads (e.g., PostgreSQL database) under steady-state usage.
- **Application Throughput:** Workload-specific throughput metrics (e.g., Requests Per Second for the NGINX web server, Transactions Per Second for the PostgreSQL database) during steady-state operation.
- **Application Latency:** Workload-specific latency metrics (e.g., Average Response Time for the NGINX web server, Average Inference Time for TensorFlow Lite) during steady-state operation.

5.2 Step-by-step implementation of container migration

To facilitate the setup and deployment of container migration using FlexiMigrate, we followed a structured step-by-step process, ensuring proper configuration and execution.

1. **Install Required Dependencies:** We installed the necessary Python packages using the following command:

```
1 #Install necessary packages
2 pip install elasticsearch os-ken docker prometheus_client bsdiff4 networkx
   ↪ grpcio tensorflow
```

This command installed the prerequisite Python libraries essential for FlexiMigrate's operation. *Elasticsearch* was used for data storage and retrieval, *os_ken* provided SDN controller functionality, and *Docker* enabled interaction with the container runtime. *Prometheus_client* facilitated metrics collection for monitoring, while *bsdiff4* supported efficient binary differencing during migration. *Networkx* was utilized for network topology analysis, *grpcio* enabled remote procedure calls, and *TensorFlow* powered the machine learning components, such as predictive migration. These dependencies collectively provided the necessary tools for managing, monitoring, and executing container migrations.

2. Configuration of Container Runtimes and SDN Controllers: We configured the container runtime and SDN controllers to enable seamless container migration by ensuring integration between the orchestration environment and the software-defined network.

- Docker Installation and Configuration: We installed and configured Docker, the container runtime, on each host machine. The following commands were used on all host machines with the Ubuntu/Debian operating system:

```

1  # Update package index
2  sudo apt-get update
3
4  # Install dependencies
5  sudo apt-get install -y apt-transport-https ca-certificates curl software-
   ↪ properties-common
6
7  # Add Docker's official GPG key
8  curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --
   ↪ dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
9
10 # Add Docker repository
11 echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings
   ↪ /docker-archive-keyring.gpg] https://download.docker.com/linux/
   ↪ ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.
   ↪ d/docker.list > /dev/null
12
13 # Install Docker
14 sudo apt-get update
15 sudo apt-get install -y docker-ce docker-ce-cli containerd.io
16
17 # Enable and start Docker service
18 sudo systemctl enable --now docker
19
20 # Verify installation
21 sudo docker info

```

- We installed Docker and its associated components on each host machine. We first updated the package index and installed the required dependencies to securely retrieve packages. Next, Docker's official GPG key was added for authentication purposes, followed by adding Docker's repository to the system. Docker was then installed, along with its CLI and container management tool, containerd.io. The service was configured to start automatically on boot and started immediately. Finally, we verified the installation by executing 'sudo docker info', confirming the proper setup of the Docker runtime on all hosts.

- os_ken SDN Controller Deployment: We deployed and configured the 'os_ken' SDN controller to manage the network traffic flow. The following commands were used on the host machine:

```

1  # Install dependencies
2  sudo apt-get install -y python3-pip
3
4  # Install os_ken
5  pip3 install os_ken
6
7  # Create simple controller (simple_switch.py)
8  cat <<EOF > simple_switch.py
9  from os_ken.base import app_manager
10 from os_ken.controller import ofp_event
11 from os_ken.lib.packet import ethernet
12 from os_ken.ofproto import ofproto_v1_3
13
14 class SimpleSwitch(app_manager.OSKenApp):
15     OFP_VERSIONS = [ofproto_v1_3.OFP_VERSION]
16
17     def __init__(self, *args, **kwargs):
18         super(SimpleSwitch, self).__init__(*args, **kwargs)
19
20     @set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
21     def packet_in_handler(self, ev):
22         pass
23 EOF
24
25 # Run controller
26 os_ken run simple_switch.py --verbose

```

- We first installed the necessary dependencies, including ‘python3-pip’. Then, we installed the ‘os_ken’ library using ‘pip3’, which is required to run the SDN controller. Next, we created a basic controller script, ‘simple_switch.py’, that defines a simple OpenFlow switch. This controller does not process packets actively in this example but can be expanded for more complex functionality. We then launched the controller using the ‘os_ken run’ command in verbose mode. This setup enabled the controller to manage network flows, adjusting paths dynamically as needed during container migration. The SDN controller is crucial for ensuring that the network topology can be altered based on the needs of the migrating containers. Next, we configured the network device, specifically an Open vSwitch (OVS) bridge, to connect to the SDN controller. We used the following commands:

```

1  # Set controller for OVS bridge
2  sudo ovs-vsctl set-controller <bridge_name> tcp:<controller_ip>:6633
3
4  # Verify connection
5  sudo ovs-vsctl show

```

- We used the ‘ovs-vsctl set-controller’ command to set the controller’s address (IP and port 6633, the default OpenFlow port) for the specified bridge. After executing this command, we verified the connection by running ‘sudo ovs-vsctl show’, which

displayed the OVS configuration. This allowed us to confirm that the bridge was successfully connected to the controller. Once the connection was established, the SDN controller was able to manage the network flows of the OVS bridge, enabling dynamic network adjustments during container migrations. To validate the setup, we used the following commands:

```

1  # Check Docker status
2  systemctl status docker
3
4  # Verify SDN controller connectivity
5  telnet <controller_ip> 6633

```

- We used the ‘systemctl status docker’ command to confirm that the Docker service was running correctly on the host machines, ensuring that the container runtime environment was operational.

Additionally, we used the ‘telnet<controller_ip> 6633’ command to attempt a basic TCP connection to the SDN controller on port 6633. A successful connection indicated that the controller was reachable and listening for OpenFlow connections,

confirming network connectivity between the hosts and the controller. These checks ensured that the basic infrastructure was in place and ready for the next phase.

3. **Define Migration Policies:** We defined migration policies based on specific conditions using Python language. An example of this is the policy ‘adaptive_load_balancing’, which considers factors such as CPU utilization, time of day, network congestion, and service type to trigger migrations.

```

1  #Define a migration policy
2  policies = [
3      {
4          'policy_name': 'adaptive_load_balancing',
5          'CONTEXT': ['source_cpu_utilization', 'destination_cpu_utilization',
6                     'time_of_day', 'network_congestion_prob', 'service_type'],
7          'CONDITIONS': '(source_cpu_utilization > 80 and
8                          ↳ destination_cpu_utilization < 50) or '
9                          ↳ '(time_of_day >= 18 and time_of_day <= 22 and
10                             ↳ service_type == "critical") or '
11                             ↳ '(network_congestion_prob < 0.2)',
12          'ACTIONS': ['allow_migration', 'set_priority("high")',
13                     'trigger_load_balancer_reconfiguration'],
14          'CONSTRAINTS': {'max_concurrent_migrations': 5, 'migration_duration':
15                          ↳ 300},
16          'PRIORITY': 2
17      }
18  ]

```

This policy defined rules for when and how container migrations should occur. The ‘CONTEXT’ specified the relevant factors (e.g., CPU utilization, time of day). ‘CONDITIONS’ set the triggers for migration (e.g., high CPU usage at the source and low usage at the destination). The ‘ACTIONS’ defined the steps to take when migration was triggered (e.g., allowing migration and adjusting priority). ‘CONSTRAINTS’ restricted migration behavior (e.g., limiting the number of concurrent migrations). Finally, the ‘PRIORITY’ ensured that policies were evaluated in the correct order.

4. **Initialize and Run the FlexiMigrate Framework:** We initialized and ran the FlexiMigrate framework by executing the following code:

```

1  # Initialize and run FlexiMigrate
2  if __name__ == '__main__':
3      flexi_migrate = FlexiMigrate(policies=policies)
4      flexi_migrate.run()

```

This step marked the core initialization of the framework. We created an instance of the ‘FlexiMigrate’ class, passing in the previously defined migration policies. The ‘flexi_migrate.run()’ method then started the FlexiMigrate framework, which began monitoring the system, evaluating the policies, and managing container migrations according to those policies. This effectively activated the entire migration management system, enabling real-time container migrations based on the defined policies.

5. **Interact with the FlexiMigrate Framework:** We interacted with the FlexiMigrate Framework by adding hosts and containers to the system and initiating migration requests based on specific policies.

- **Adding Hosts and Containers:** The following codes were used to add hosts and containers to the FlexiMigrate framework.

```

1  # Add host1
2  host1 = Host(host_id='host1', total_cpu=16, total_memory=32768,
    ↪ total_storage=1000)
3
4  # Add host2
5  host2 = Host(host_id='host2', total_cpu=16, total_memory=32768,
    ↪ total_storage=1000)
6
7  # Add hosts to FlexiMigrate
8  flexi_migrate.migration_engine.resource_optimizer.hosts = {
9      host1.host_id: host1,
10     host2.host_id: host2
11 }
12
13 # Add container1
14 container1 = Container(container_id='container1', image='nginx:latest',
15                        cpu_limit=4, memory_limit=2048, storage_limit=50)
16 container1.host = 'host1'
17 host1.containers.append(container1)
18
19 # Add container1 to FlexiMigrate
20 flexi_migrate.migration_engine.resource_optimizer.containers = {
21     container1.container_id: container1
22 }

```

- These codes demonstrated how to inform FlexiMigrate about the available resources and running containers. 'Host' objects were created to represent physical or virtual machines, with their resource capacities (CPU, memory, storage) specified. 'Container' objects were created to represent the running containers, with their resource limits and the hosts they were running on. These objects were then added to the 'resource_optimizer' within FlexiMigrate's 'migration_engine'. This allowed FlexiMigrate to track resource usage and make informed decisions about container placement and migration.

the actual migration process. This illustrated both manual migration initiation and the crucial policy enforcement step.

5.3 Experimental results

In this section, we present the results of the experiments conducted to evaluate the performance of the FlexiMigrate framework.

```

1  # Create a migration request
2  migration_request = MigrationRequest(
3      container_id='container1',
4      source_host=flexi_migrate.resource_optimizer.hosts['host1'],
5      destination_host=flexi_migrate.resource_optimizer.hosts['host2'],
6      migration_type=MigrationStrategy.LIVE_MIGRATION
7  )
8  if flexi_migrate.decision_engine.enforce_policies(migration_request):
9      flexi_migrate.migration_manager.add_migration_request(
    ↪ migration_request)

```

- Requesting Migrations: The following codes were used to request a container migration. These codes demonstrated how to request a container migration. A 'MigrationRequest' object was created, specifying the container to be migrated, the source and destination hosts, and the migration type (in this case, 'LIVE_MIGRATION', implying minimal downtime). The 'decision_engine' then checked this request against the defined policies. If the policies allowed the migration, the request was added to the 'migration_manager', which then orchestrated

5.3.1 Evaluation of FlexiMigrate compared to existing frameworks

Table 7 presents the experimental results for the migration of a single container running the NGINX web server between edge nodes. The results demonstrate that the FlexiMigrate framework consistently outperforms the ProKube, LIMOCE, and CRIU-based frameworks across all evaluated performance metrics. As shown in Table 7, FlexiMigrate achieves a 15.8% reduction in migration time compared to ProKube, a 28.9% reduction compared to LIMOCE, and a

Table 7 Experimental Results for the Single Container Migration

Performance Metrics	FlexiMigrate	ProKube	LIMOCE	CRIU-based
Total Migration Time (s)	3.2	3.8	4.5	5.7
Downtime During Migration (ms)	52	75	98	245
Network Overhead (MB)	78	95	112	156
Avg. CPU Usage (%)	35	38	42	48
Peak Memory Usage (MB)	450	485	520	580

43.9% reduction compared to CRIU-based migration. This significant improvement in migration speed highlights the efficiency of FlexiMigrate's innovative nested container architecture and advanced checkpointing mechanism. In terms of downtime, FlexiMigrate shows impressive reductions of 30.7% compared to ProKube, 46.9% compared to LIMOCE, and 78.8% compared to CRIU-based migration. These results underscore the effectiveness of FlexiMigrate's state transfer and checkpointing mechanism in maintaining service availability during migration. Network overhead is another area where FlexiMigrate excels, with reductions of 17.9% compared to ProKube, 30.4% compared to LIMOCE, and 50% compared to CRIU-based migration. This lower network utilization demonstrates FlexiMigrate's superior efficiency in data transfer during the migration process. Resource utilization metrics further emphasize FlexiMigrate's advantages. For average CPU usage, FlexiMigrate consumes 7.9% less CPU compared to ProKube, 16.7% less compared to LIMOCE, and 27.1% less compared to CRIU-based frameworks. These results indicate that FlexiMigrate places a significantly lighter load on system resources during migration. Lastly, FlexiMigrate exhibits impressive memory efficiency, with a 7.2% reduction in peak memory usage compared to ProKube, a 13.5% reduction compared to LIMOCE, and a 22.4% reduction compared to CRIU-based migration. This further demonstrates FlexiMigrate's resource-efficient design and implementation.

Table 8 presents the experimental results for the migration of a multi-container application, which includes both NGINX and PostgreSQL services. These results clearly indicate that FlexiMigrate outperforms the other frameworks, providing notable improvements across all performance metrics. FlexiMigrate reduces the total migration time by 14.3% compared to ProKube, 30.4% compared to LIMOCE, and 46.2% compared to CRIU-based migration. This reduction in migration time highlights the efficiency of FlexiMigrate in handling complex multi-container

Table 8 Experimental Results for the Multi-Container Application Migration

Performance Metrics	FlexiMigrate	ProKube	LIMOCE	CRIU-based
Total Migration Time (s)	7.8	9.1	11.2	14.5
Downtime During Migration (ms)	124	182	287	518
Network Overhead (MB)	215	245	298	384
Avg. CPU Usage (%)	52	57	61	68
Peak Memory Usage (MB)	780	835	890	980

applications. Furthermore, FlexiMigrate shows remarkable reductions in downtime, decreasing service unavailability by 31.9% compared to ProKube, 56.8% compared to LIMOCE, and 76.1% compared to CRIU-based migration. This ensures better service continuity and significantly minimizes disruption during the migration process. Regarding network overhead, FlexiMigrate demonstrates a 12.2% reduction in network usage compared to ProKube, a 27.9% reduction compared to LIMOCE, and a 44.0% reduction compared to CRIU-based frameworks. This reduced network strain highlights FlexiMigrate's superior optimization in managing network resources during the migration process. In terms of resource utilization, FlexiMigrate is more CPU-efficient, consuming 8.8% less CPU compared to ProKube, 14.8% less compared to LIMOCE, and 23.5% less compared to CRIU-based frameworks. Such efficiency in CPU usage allows for smoother system operation and less resource competition during the migration. Finally, FlexiMigrate showcases its effectiveness in memory usage, with a 6.6% reduction in peak memory usage compared to ProKube, a 12.4% reduction compared to LIMOCE, and a 20.4% reduction compared to CRIU-based migration. These results demonstrate that FlexiMigrate is well-suited for resource-constrained environments, providing an optimal solution for multi-container application migrations.

Table 9 outlines the results of application migration experiments conducted under network congestion conditions. FlexiMigrate consistently delivers superior performance across all performance metrics. With a total migration time of 12.4 seconds, it completes migrations 16.2% faster than ProKube, 33.7% quicker than LIMOCE, and 50.8% more swiftly than the CRIU-based framework. This reduction is crucial for ensuring that applications remain responsive, even when the network is under strain. Downtime is another area where FlexiMigrate outperforms its competitors. The recorded downtime of 198 milliseconds translates to a

Table 9 Experimental Results for Migration under Network Congestion

Performance Metrics	FlexiMigrate	ProKube	LIMOCE	CRIU-based
Total Migration Time (s)	12.4	14.8	18.7	25.2
Downtime During Migration (ms)	198	275	456	892
Network Overhead (MB)	267	312	389	512
Avg. CPU Usage (%)	58	62	67	75
Peak Memory Usage (MB)	620	665	710	790

decrease of 28.0% compared to ProKube, 56.6% compared to LIMOCE, and 77.8% compared to CRIU-based migrations. Such minimal service interruptions enhance operational continuity during the migration process. Additionally, FlexiMigrate is highly efficient in managing network traffic. The network overhead results show that it uses 14.4% less data than ProKube, 31.4% less than LIMOCE, and 47.9% less than CRIU-based frameworks. This reduction in bandwidth usage is particularly beneficial under congestion, allowing for smoother data transfers without overburdening the network. Looking at CPU utilization, FlexiMigrate again proves its efficiency. It uses 6.5% less CPU than ProKube, 13.4% less than LIMOCE, and 22.7% less than CRIU-based frameworks, allowing the system to maintain higher overall performance during the migration process. Lastly, the peak memory usage results emphasize FlexiMigrate's resource efficiency, with 6.8% less memory consumed compared to ProKube, 12.7% less than LIMOCE, and 21.5% less than CRIU-based migrations. This reduced memory demand makes it an ideal solution for environments with limited resources, ensuring that applications can migrate without overwhelming system capacity.

5.3.2 Evaluation of FlexiMigrate's nested container architecture

Table 10 presents the experimental results for migrating a basic stateful web application container from a Kubernetes (K8s) node to a Docker Swarm node across two deployment strategies.

In the standard deployment, the migration failed due to several inherent incompatibilities between the networking models, state management approaches, and API structures of Docker Swarm and Kubernetes (K8s). This results in migration failure and requires substantial manual intervention. Specifically, migration attempts necessitate extensive

Table 10 Experimental Results for Migrating a Basic Stateful Web Application

Performance Metrics	Standard Deployment	Flexi-Migrate
Migration Feasibility	Failure	Success
Manual Intervention Complexity	High ¹	Low ²
Total Migration Time (s)	N/A	4.1
Downtime During Migration (ms)	N/A	68
Network Overhead (MB)	N/A	78
Avg. CPU Usage (%)	N/A	35
Peak Memory Usage (MB)	N/A	450

¹ Due to extensive reconfiguration, API mapping, and state management.

² Due to the automated process

reconfiguration of network settings, API mappings, and manual state management, adding significant complexity. The need for such intricate adjustments makes the migration unfeasible without considerable custom engineering efforts, highlighting the limitations of direct migration between heterogeneous orchestration platforms. In contrast, FlexiMigrate successfully completes the migration. The nested container architecture plays a crucial role in abstracting platform-specific details, enabling a seamless transfer between heterogeneous orchestration platforms. The entire process is automated, with minimal manual intervention—only the initial migration request is required—demonstrating the framework's efficiency and user-friendliness. The total migration time for FlexiMigrate is recorded at 4.1 seconds, reflecting the framework's ability to perform the migration with minimal overhead. In contrast, the standard deployment does not provide a comparable migration time as the migration is infeasible. Additionally, the downtime during the FlexiMigrate migration is just 68 ms, highlighting the framework's ability to maintain near-zero service interruption—a key advantage for environments requiring high availability. Furthermore, the network overhead introduced by FlexiMigrate during the migration process is measured at 78 MB, indicating efficient state and container packaging mechanisms within the framework. Average CPU usage during the migration remains moderate at 35%, demonstrating that the framework operates without imposing excessive processing burden on the system. The peak memory usage is recorded at 450 MB, reflecting the memory consumption associated with container encapsulation and migration processes, which remains well within acceptable limits for typical edge nodes. These results underscore the significance of FlexiMigrate's nested container architecture in enabling orchestrator-agnostic live container migration.

Table 11 presents the experimental results on system overhead in the FlexiMigrate framework with the nested container architecture, compared to a standard deployment. While the nested container architecture enables seamless

Table 11 System Overhead of Nested Container Architecture in FlexiMigrate

Performance Metrics	Workload	Standard Deployment	FlexiMigrate	Overhead
Avg. CPU Usage	NGINX	5.0%	5.3%	+6.0%
	PostgreSQL	15.0%	16.0%	+6.7%
	TensorFlow Lite	25.0%	26.5%	+6.0%
Avg. Memory Usage	NGINX	50 MB	55 MB	+10.0%
	PostgreSQL	250 MB	268 MB	+7.2%
	TensorFlow Lite	150 MB	165 MB	+10.0%
Disk I/O Throughput	PostgreSQL	85 ops/sec	83 ops/sec	−2.4%
Application Throughput	NGINX	1200 req/sec	1170 req/sec	−2.5%
	PostgreSQL	350 TPS	340 TPS	−2.9%
Application Latency	NGINX	25 ms	27 ms	+8.0%
	TensorFlow Lite	150 ms	158 ms	+5.3%

container migration across heterogeneous orchestration platforms, it introduces a measurable but generally minor overhead relative to standard deployments.

The average CPU usage increased moderately across all workloads, with the NGINX web server, PostgreSQL database, and TensorFlow Lite exhibiting rises of approximately 6%. This slight increase is primarily attributed to the additional management operations performed within the parent container, which encapsulates and supervises the nested application container. Similarly, average memory usage shows a modest increase, ranging from 7.2% to 10% across workloads, reflecting the memory overhead required to maintain the parent container and execute the associated FlexiMigrate services. For disk-intensive workloads, such as PostgreSQL database, the disk I/O throughput experienced a slight reduction of 2.4%. This can be attributed to the additional I/O operations needed to ensure consistent state management within the nested container framework. Application throughput for both the NGINX web server (measured in requests per second) and the PostgreSQL database (measured in transactions per second) exhibited a small decline, with decreases of 2.5% and 2.9%, respectively. These minor reductions are likely caused by the runtime overhead imposed by the extra layer of abstraction and synchronization within the nested architecture. Conversely, application latency for both the NGINX web server and the TensorFlow Lite increased slightly, by 8.0% and 5.3% respectively. This latency increase stems from the additional steps involved in managing the application state and ensuring consistency between the parent and nested containers during normal operations. Despite these measurable overheads, the results demonstrate that the performance cost of FlexiMigrate's nested container architecture

remains relatively small and acceptable for practical use. The observed trade-offs—moderate increases in CPU and memory usage, slight reductions in throughput, and minor increases in latency—are justified by the significant benefits provided by FlexiMigrate. These include orchestrator-agnostic live migration, minimal service downtime, and reduced migration complexity across heterogeneous cloud and edge environments. Overall, FlexiMigrate achieves a highly favorable balance between system performance and operational flexibility, validating the practical effectiveness of its nested container architecture.

6 Discussion

As shown in Sect. 5.3, the FlexiMigrate framework consistently outperforms existing container migration frameworks (i.e., ProKube, LIMOCE, and CRIU-based frameworks) across various experimental scenarios, including Single Container Migration, Multi-Container Application Migration, and Migration under Network Congestion. These improvements stem from the following key functionalities:

- *Delta-Based Checkpointing*: By continuously tracking changes through the *Delta Tracker* module within the *State Synchronizer* component, FlexiMigrate eliminates redundant data transmission. Instead of transferring full container snapshots, it transmits only incremental state changes (e.g., modified memory pages, filesystem deltas). This approach significantly reduces total migration time, service downtime, network overhead, CPU utilization, and memory usage.
- *Nested Container Architecture*: By encapsulating application containers within parent containers, FlexiMigrate abstracts orchestration-specific configurations and APIs, enabling seamless migration across heterogeneous platforms without requiring reconfiguration. This decoupling simplifies the management of multi-container applications by treating the entire application stack as a single migratable unit. Additionally, it reduces CPU utilization by offloading orchestration-specific tasks to the parent container.
- *SDN-Driven Network Adaptation*: The integration of SDN enables FlexiMigrate to dynamically reconfigure network routes and reallocate IP addresses in real time, preventing disruptions and ensuring seamless connectivity. This functionality reduces service downtime by maintaining continuous network connectivity throughout migration and optimizes network resource utilization. By dynamically managing network paths, prioritizing critical data transfers, and avoiding congested routes, FlexiMigrate reduces network overhead.

- *ML-Driven Migration Decision-Making*: The use of ML for optimizing resource allocation during live container migration enables FlexiMigrate to adapt to changing conditions. It makes intelligent migration decisions that balance workload distribution and minimize computational costs. This functionality reduces CPU utilization by dynamically allocating resources based on real-time system conditions and workload analysis. Additionally, the intelligent selection of migration destinations enhances workload balancing and improves overall system performance.

Despite these advantages, implementing FlexiMigrate in multi-container environments introduces several challenges that require further consideration:

1. Coordinating state synchronization across interdependent containers (e.g., databases and web servers) requires precise dependency management to avoid data inconsistencies during migration.
2. Network reconfiguration complexity increases with the number of containers, as dynamic IP assignments and service discovery mechanisms must be seamlessly updated to maintain connectivity.
3. Resource contention may arise when migrating multiple containers simultaneously, necessitating advanced scheduling algorithms to balance bandwidth and compute resources.
4. Ensuring fault tolerance in large-scale migrations demands robust rollback mechanisms to handle partial failures without compromising application integrity. Addressing these challenges is critical for scaling FlexiMigrate to production-grade multi-container ecosystems.

7 Conclusion and Future Work

In this paper, we introduced FlexiMigrate, a framework designed to address key limitations in live container migration within cloud and edge computing environments. By tackling challenges such as orchestration platform dependency, platform heterogeneity, state transfer overhead, network disruptions, service downtime, and suboptimal migration decision-making, FlexiMigrate offers innovative solutions that enhance migration efficiency and reduce system overhead. Through its unique combination of a nested container architecture, advanced checkpointing, SDN-based network management, and ML-driven migration decision-making, FlexiMigrate offers a flexible and efficient approach to container migration.

While the experimental results showed that the FlexiMigrate framework introduces a minor overhead compared to standard deployment due to the use of a nested container architecture, they also highlight the clear advantages of FlexiMigrate over existing solutions such as ProKube, LIMOCE, and CRIU-based frameworks. Across single-container, multi-container, and network-congested migration scenarios, FlexiMigrate consistently achieved faster migration times, significantly lower service downtime, and reduced resource consumption—particularly in terms of network overhead, CPU usage, and memory consumption. These results affirm the framework’s capability to efficiently address real-world migration challenges.

While FlexiMigrate has shown promising results, future research will explore several avenues to further enhance its performance and applicability:

- *Integration of P4, NFV, and Network Slicing*: These technologies could be integrated into FlexiMigrate to enhance network management and adaptability in dynamic environments. Future research will explore how P4 for programmable networking, NFV for virtualized network functions, and network slicing for resource isolation can be leveraged—particularly in conjunction with the MikroTik Switch—to improve migration efficiency and flexibility.
- *Scalability for Large-Scale Deployments*: One key direction is improving the scalability of FlexiMigrate for large-scale containerized applications deployed across geographically distributed cloud and edge infrastructures. This will ensure efficient migration even in highly complex and distributed environments.
- *Advanced Machine Learning for Adaptive Migration*: Integrating more sophisticated machine learning models to predict and adapt to workload fluctuations in real time could significantly improve migration decision-making. This would enhance FlexiMigrate’s ability to optimize resource allocation dynamically.
- *Robustness Under Failure Scenarios*: To ensure FlexiMigrate’s resilience, we plan to evaluate its performance under various failure conditions, such as partial network outages. This will help develop mechanisms for fault-tolerant migration in highly dynamic and unpredictable environments.
- *Security Enhancements for Live Container Migration*: Future work will investigate the security aspects of live container migration, particularly in untrusted or multi-tenant environments. This includes exploring techniques to strengthen data privacy and integrity guarantees during migration.

Author Contributions All authors contributed to the study. SHA and MM were responsible for conceptualization and methodology. SHA

and MM wrote the original draft, while AS and SHE reviewed and edited the manuscript. MM supervised the work. All authors read and approved the final version of the manuscript.

Funding There is no funding for this research.

Declarations

Conflicts of Interest The authors have no relevant financial or non-financial interests to disclose. The authors declare no conflict of interest.

Ethics approval Not applicable.

Consent to participate Not applicable.

Consent for publication Not applicable.

Code availability The source code is available online via this link: <https://github.com/ahmadpanah/FlexiMigrate-Framework>

References

- Asim, M., Wang, Y., Wang, K., Huang, P.-Q.: A review on computational intelligence techniques in cloud and edge computing. *IEEE Trans. Emerg. Top. Comput. Intell* **4**(6), 742–763 (2020)
- Cao, K., Liu, Y., Meng, G., Sun, Q.: An overview on edge computing research. *IEEE access* **8**, 85714–85728 (2020)
- Moghadam, M.H., Babamir, S.M., Mirabi, M.: A multi-objective optimization model for data-intensive workflow scheduling in data grids. In: *Proceedings of the IEEE 41st Conference on Local Computer Networks Workshops (LCN Workshops)*, pp. 25–33 (2016)
- Hfer, C.N., Karagiannis, G.: Cloud computing services: taxonomy and comparison. *J. Internet Serv. Appl.* **2**, 81–94 (2011)
- Cheraghlo, M.N., Khadem-Zadeh, A., Haghparsat, M.: A survey of fault tolerance architecture in cloud computing. *J. Netw. Comput. Appl.* **61**, 81–92 (2016)
- Ghorashi, H., Mirabi, M.: An effective task scheduling framework for cloud computing using NSGA-II. *J. Adv. Comput. Eng. Technol.* **6**(3), 151–160 (2020)
- Xu, M., Fu, Z., Ma, X., Zhang, L., Li, Y., Qian, F., Wang, S., Li, K., Yang, J., Liu, X.: From Cloud to Edge: A First Look at Public Edge Platforms. In: *Proceedings of the 21st ACM Internet Measurement Conference*, pp. 37–53 (2021)
- Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. *IEEE Internet Things J.* **3**(5), 637–646 (2016)
- Mansouri, Y., Babar, M.A.: A review of edge computing: Features and resource virtualization. *J. Parallel Distrib. Comput.* **150**, 155–183 (2021)
- Zangaraki, S., Mirabi, M., Erfani, S.H., Sahafi, A.: SecShield: An IoT access control framework with edge caching using software defined network. *Peer-to-Peer Netw. Appl.* **18**, 1–17 (2025)
- Bentaleb, O., Belloum, A.S., Sebaa, A., El-Maouhab, A.: Containerization technologies: Taxonomies, applications and challenges. *The J. Supercomput.* **78**(1), 1144–1181 (2022)
- Watada, J., Roy, A., Kadikar, R., Pham, H., Xu, B.: Emerging trends, techniques and open issues of containerization: A review. *IEEE Access* **7**, 152443–152472 (2019)
- Pieter-Jan, M., Volckaert, B., Ongenae, V., Turck, F.D.: Resource management in a containerized cloud: Status and challenges. *J. Netw. Syst. Manag.* **28**, 197–246 (2020)
- He, T., Buyya, R.: A taxonomy of live migration management in cloud computing. *ACM Comput. Surv.* **56**(3), 1–33 (2023)
- Benjaponpitak, T., Karakate, M., Sripanidkulchai, K.: Enabling Live Migration of Containerized Applications Across Clouds. In: *Proceedings of the IEEE INFOCOM*, pp. 2529–2538 (2020)
- Lohumi, Y., Srivastava, P., Gangodkar, D., Tripathi, V.: Recent Trends, Issues and Challenges in Container and VM Migration. In: *Proceedings of the International Conference on Computer Science and Emerging Technologies*, pp. 1–5 (2023)
- Watada, J., Roy, A., Kadikar, R., Pham, H., Xu, B.: Emerging trends, techniques and open issues of containerization: A review. *IEEE Access* **7**(3), 152443–152472 (2019)
- Rouholamini, S.R., Mirabi, M., Farazkish, R., Sahafi, A.: Proactive self-healing techniques for cloud computing: A systematic review. *Concurrency and Computation: Practice and Experience* **36**, 8246 (2024)
- Ali, B., Golec, M., Gill, S.S., Cuadrado, F., Uhlig, S.: Prokube: Proactive kubernetes orchestrator for inference in heterogeneous edge computing. *Int. J. Netw. Manag.*, 2298 (2024)
- Tomarchio, O., Calcaterra, D., Modica, G.: Cloud resource orchestration in the multi-cloud landscape: a systematic review of existing frameworks. *J. Cloud Comput.*, **9** (2020) <https://doi.org/10.1186/s13677-020-00194-7>
- Caballer, M., Zala, S., Garcia, Á.L., Molto, G., Fernandez, P.O., Velten, M.: Orchestrating complex application architectures in heterogeneous clouds. *Journal of Grid Computing* **16**, 3–18 (2017) <https://doi.org/10.1007/s10723-017-9418-y>
- Castellano, G., Esposito, F., Risso, F.: A service-defined approach for orchestration of heterogeneous applications in cloud/edge platforms. *IEEE Transactions on Network and Service Management* **16**, 1404–1418 (2019) <https://doi.org/10.1109/TNSM.2019.2941639>
- Yang, S., Ren, Y., Zhang, J., Guan, J., Li, B.: KubeHic: Performance-aware container orchestration on heterogeneous-isa architectures in cloud-edge platforms. 2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCLOUD/SocialCom/SustainCom), 81–91 (2021) <https://doi.org/10.1109/ISPA-BDCLOUD-SocialCom-SustainCom52081.2021.00025>
- Fernando, D., Ternier, J., Yang, P., Gopalan, K.: V-recover: Virtual machine recovery when live migration fails. *IEEE Transactions on Cloud Computing* **11**, 3289–3300 (2023) <https://doi.org/10.1109/TCC.2023.3282466>
- Cheng, X., Wang, Z., Li, Y., Liu, Z., Tang, M.: Research on optimization model of container live migration. 2023 IEEE 7th Information Technology and Mechatronics Engineering Conference (ITOEC) **7**, 1538–1542 (2023) <https://doi.org/10.1109/ITOEC57671.2023.10291208>
- Akinola, O.I., Olaniyi, O.O., Ogungbemi, O.S., Oladoyinbo, O.B., Olisa, A.O.: Resilience and recovery mechanisms for software-defined networking (sdn) and cloud networks. *Journal of Engineering Research and Reports* (2024). <https://doi.org/10.9734/jerr/2024/v26i81234>
- Cao, R., Tang, Z., Li, K., Li, K.: Hmgowm: A hybrid decision mechanism for automating migration of virtual machines. *IEEE Transactions on Services Computing* **14**, 1397–1410 (2021) <https://doi.org/10.1109/TSC.2018.2873694>
- Brahmam, M.G., R., V.: Vmmisd: An efficient load balancing model for virtual machine migrations via fused metaheuristics with iterative security measures and deep learning optimizations. *IEEE Access* **12**, 39351–39374 (2024) <https://doi.org/10.1109/ACCESS.2024.3373465>
- He, Z., Li, L., Lin, Z., Dong, Y., Qin, J., Li, K.: Joint optimization of service migration and resource allocation in mobile edge-cloud

- computing. *Algorithms* 17, 370 (2024) <https://doi.org/10.3390/a17080370>
30. Mansouri, Y., Toosi, A.N., Babar, M.A.: Container orchestration platforms: A comparative study. *IEEE Trans. Cloud Comput.* **10**(4), 2715–2731 (2022)
 31. Pahl, C., Brogi, A., Soldani, J., Jamshidi, P.: Cloud container technologies: a state-of-the-art review. *IEEE Transactions on Cloud Computing* **7**(3), 677–692 (2019)
 32. Bellavista, P., Zanni, A.: Feasibility of fog computing deployment based on docker containerization over raspberrypi. In: *Proceedings of the 18th Inter. Confe. on Dist. Comput. Network.* 1–10 (2017)
 33. Nadgowda, S., Suneja, S., Bila, N., Isci, C.: Voyager: Complete container state migration. In: *Proceedings of the 37th Inter. Confer. Dist. Comput. Syst. (ICDCS)*, pp. 2137–2142 (2017)
 34. Das, R., Sidhanta, S.: Live migration of containers in the edge. *SN Comput. Sci.* **4**(5), 479 (2023)
 35. Bhardwaj, A., Krishna, C.R.: A container-based technique to improve virtual machine migration in cloud computing. *IETE J. Res.* **68**(1), 401–416 (2022)
 36. Sharma, S., Chawla, M.: A three phase optimization method for precopy based vm live migration. *SpringerPlus* **5**, 1–24 (2016)
 37. Wu, T.-Y., Guizani, N., Huang, J.-S.: Live migration improvements by related dirty memory prediction in cloud computing. *J. Netw. Comput. Appl.* **90**, 83–89 (2017)
 38. Belgacem, A., Mahmoudi, S., Ferrag, M.A.: A machine learning model for improving virtual machine migration in cloud computing. *J. Supercomput.* **79**, 9486–9508 (2023)
 39. Jin, X., He, S., Chen, Y.: Container migration for edge computing in industrial internet with joint latency reduction and reliability enhancement. *Sci. Rep.* **14**, 25394 (2024)
 40. Govindaraj, K., Artemenko, A.: Container live migration for latency critical industrial applications on edge computing. In: *Proceedings of the 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*, pp. 83–90 (2018)
 41. Al-Dhuraibi, Y., Paraiso, F., Djarallah, N., Merle, P.: Autonomic vertical elasticity of docker containers with elasticdocker. In: *Proceedings of the IEEE 10th International Conference on Cloud Computing (CLOUD)*, pp. 472–479 (2017)
 42. Tran, M.-N., Vu, X.T., Kim, Y.: Proactive stateful fault-tolerant system for kubernetes containerized services. *IEEE Access* **10**, 102181–102194 (2022)
 43. Machen, A., Wang, S., Leung, K.K., Ko, B.J., Salonidis, T.: Live service migration in mobile edge clouds. *IEEE Wirel. Commun.* **25**(1), 140–147 (2017)
 44. Ma, L., Yi, S., Carter, N., Li, Q.: Efficient live migration of edge services leveraging container layered storage. *IEEE Trans. Mob. Comput.* **18**(9), 2020–2033 (2019)
 45. Yin, L., Li, P., Luo, J.: Smart contract service migration mechanism based on container in edge computing. *J. Parallel Distrib. Comput.* **152**, 157–166 (2021)
 46. Chanikaphon, T., Salehi, M.A.: UMS: Live migration of containerized services across autonomous computing systems. In: *GLOBECOM 2023-2023 IEEE Glob Commun. Conf.* 467–472 (2023)
 47. Ray, P.P.: An overview of WebAssembly for IoT: Background, tools, state-of-the-art, challenges, and future directions. *Future Internet* **15**(8), 275 (2023)
 48. Zhang, Y., Cao, S., Wang, H., Chen, Z., Luo, X., Mu, D., Ma, Y., Huang, G., Liu, X.: Characterizing and detecting webassembly runtime bugs. *ACM Trans. Softw. Eng. Meth.* **33**(2), 1–29 (2023)
 49. Rahim, J.A., Nordin, R., Amodu, O.A.: Open-source software defined networking controllers: State-of-the-art, challenges and solutions for future network providers. *Comput. Mater. Contin.* **80**(1), 155–179 (2024)
 50. McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., Shenker, S., Turner, J.: OpenFlow: enabling innovation in campus networks. *ACM SIGCOMM comput. commun. Rev.* **38**(2), 69–74 (2008)
 51. Lara, A., Kolasani, A., Ramamurthy, B.: Network innovation using OpenFlow: A survey. *IEEE commun. surv. tutor.* **16**(1), 493–512 (2013)
 52. Minhas, S., Jaswal, R., Sharma, A., Singla, S.: Revolutionizing networking: A comprehensive overview of intent-based networking. In: *Proceedings of the International Conference on Emerging Innovations and Advanced Computing (INNOCOMP)*, 463–468 (2024)
 53. Leivadeas, A., Falkner, M.: A survey on intent-based networking. *IEEE Commun. Surv. Tutor.* **25**(1), 625–655 (2022)
 54. Petrov, I.: Frameworks and protocols for achieving interoperability in multi-cloud networking: A technical perspective. *MZ Comput. J.* **5**(1), 1–5 (2024)
 55. Zakurdaev, G., Ismail, M., Lung, C.-H.: Dynamic on-demand virtual extensible lan tunnels via software-defined wide area networks. In: *Proceedings of the IEEE 12th Annual Computing and Communication Workshop and Conference (CCWC)*, pp. 995–1000 (2022)
 56. Bavani, K.P.D., Kalaimannan, E.: Comprehensive survey of implementing multiple controllers in a software-defined network (SDN). *Software-Defined Network Frameworks*, 155–179 (2024)
 57. Kazemian, M.M., Mirabi, M.: Controller placement in software defined networks using multi-objective antlion algorithm. *J. Supercomput.* **78**(4), 5626–5649 (2022)
 58. Kim, H., Feamster, N.: Improving network management with software defined networking. *IEEE Commun. Mag.* **51**(2), 114–119 (2013)
 59. Kirkpatrick, K.: Software-defined networking. *Communications of the ACM* **56**(9), 16–19 (2013)
 60. Mohamed, A., Hamdan, M., Khan, S., Abdelaziz, A., Babiker, S.F., Imran, M., Marsono, M.N.: Software-defined networks for resource allocation in cloud computing: A survey. *Comput. Netw.* **195**, 108151 (2021)
 61. Priyadarsini, M., Bera, P.: Software defined networking architecture, traffic management, security, and placement: A survey. *Comput. Netw.* **192**, 108047 (2021)
 62. Xiao, H., Li, Z., Zhai, E., Xu, T., Li, Y., Liu, Y., Zhang, Q., Liu, Y.: Towards web-based delta synchronization for cloud storage services. In: *Proceedings of the 16th USENIX Conference on File and Storage Technologies (FAST 18)*, pp. 155–168 (2018)
 63. Mallu, L., Ezhilarasie, R.: Live migration of virtual machines in cloud environment: A survey. *Indian J. Sci. Technol.* **8**(S9), 326–332 (2015)
 64. Ruan, Y., Cao, Z., Cui, Z.: Pre-filter-copy: Efficient and self-adaptive live migration of virtual machines. *IEEE Syst. J.* **10**(4), 1459–1469 (2014)
 65. Zhong, Y., Xu, J., Li, Q., Zhang, H., Liu, F.: Memory state transfer optimization for pre-copy based live vm migration. In: *Proceedings of the IEEE Workshop on Advanced Research and Technology in Industry Applications (WARTIA)*, pp. 290–293 (2014)
 66. Hines, M.R., Deshpande, U., Gopalan, K.: Post-copy live migration of virtual machines. *ACM SIGOPS oper. syst. Rev.* **43**(3), 14–26 (2009)
 67. Elsaid, M.E., Abbas, H.M., Meinel, C.: Virtual machines pre-copy live migration cost modeling and prediction: a survey. *Distributed and Parallel Databases* **40**(2), 441–474 (2022)
 68. Ashour, M., Mahdiyar, A.: A comprehensive state-of-the-art survey on the recent modified and hybrid analytic hierarchy process approaches. *Applied Soft Computing*, 111014 (2023)
 69. Vaidya, O.S., Kumar, S.: Analytic hierarchy process: An overview of applications. *Eur. J. oper. Res.* **169**(1), 1–29 (2006)
 70. Vargas, L.G.: An overview of the analytic hierarchy process and its applications. *Eur. J. Oper. Res.* **48**(1), 2–8 (1990)

71. Chunlin, L., Hammad-Ur-Rehman, Q.: Adaptive threshold detection based on current demand for efficient utilization of cloud resources. In: *Proceedings of the IEEE 4th International Conference on Computer and Communication Systems (ICCCS)*, pp. 341–346 (2019)
72. Ni, W., Zhang, Y., Li, W.W.: An optimal strategy for resource utilization in cloud data centers. *IEEE Access* **7**, 158095–158112 (2019)
73. Manzali, Y., Elfar, M.: Random forest pruning techniques: a recent review. *Operations research forum* **4**(2), 43 (2023)
74. Khwaja, S., Alshayeb, M.: Survey on software design-pattern specification languages. *ACM Comput. Surv.* **49**(1), 1–35 (2016)
75. Yu, W., Wang, Y., Pu, C.: A dynamic two-phase commit protocol for self-adapting services. In: *Proceedings of the International Conference on Services Computing*, pp. 7–15 (2004)
76. Boutros, B.S., Desai, B.C.: A two-phase commit protocol and its performance. In: *Proceedings of the 7th International Conference on Database and Expert Systems Applications*, pp. 100–105 (1996)
77. Desai, B.C., Boutros, B.S.: Performance of a two-phase commit protocol. *Information and Software Technology* **38**(9), 581–599 (1996)
78. Huang, Z., Wu, S., Jiang, S., Jin, H.: Fastbuild: Accelerating docker image building for efficient development and deployment of container. In: *Proceedings of the 35th Symposium on Mass Storage Systems and Technologies (MSST)*, pp. 28–37 (2019)
79. Zhao, N., Lin, M., Albahar, H., Paul, A.K., Huan, Z., Abraham, S., Chen, K., Tarasov, V., Skourtis, D., Anwar, A.: An end-to-end high-performance deduplication scheme for docker registries and docker container storage systems. *ACM Transactions on Storage* **20**(3), 1–35 (2024)
80. Chen, Z.: Algorithm-based recovery for iterative methods without checkpointing. In: *Proceedings of the 20th International Symposium on High Performance Distributed Computing*, pp. 73–84 (2011)
81. Broder, A.Z.: Some applications of rabin's fingerprinting method. In: *Sequences II: Methods in Communication, Security, and Computer Science*, pp. 143–152 (1993)
82. Staudemeyer, R.C., Morris, E.R.: Understanding LSTM—a tutorial into long short-term memory recurrent neural networks. *arXiv preprint arXiv:1909.09586* (2019)
83. Fan, H., Sun, C., Long, J., Wang, S., Zeng, F.: A novel method for solving the multi-commodity flow problem on evolving networks. *Comput. Netw.* **247**, 110451 (2024)
84. Ongaro, D., Ousterhout, J.: In search of an understandable consensus algorithm. In: *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pp. 305–319 (2014)
85. Huang, D., Ma, X., Zhang, S.: Performance analysis of the Raft consensus algorithm for private blockchains. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* **50**(1), 172–181 (2019)
86. Netto, H., Oliveira, C.P., Oliveira Rech, L., Alchieri, E.: Incorporating the Raft consensus protocol in containers managed by Kubernetes: an evaluation. *Int. J. Parallel, Emergent Distrib. Syst.* **35**(4), 433–453 (2020)
87. Ha, M., Kim, S.-H.: CCoW: Optimizing copy-on-write considering the spatial locality in workloads. *Electronics* **11**(3), 461 (2022)
88. Smith, J.M., Jr, G.Q.M.: Effects of Copy-on-Write memory management on the response time of UNIX fork operations. *Computing Systems* **1**(3), 255–278 (1988)
89. Deng, Y., Huang, X., Song, L., Zhou, Y., Wang, F.Z.: Memory deduplication: An effective approach to improve the memory system. *J. Inf. Sci. Eng.* **33**(5), 1103–1120 (2017)
90. Sharma, P., Kulkarni, P.: Singleton: system-wide page deduplication in virtual environments. In: *Proceedings of the 21st International Symposium on High-Performance Parallel and Distributed Computing*, pp. 15–26 (2012)
91. Ibtesham, D., Ferreira, K.B., Arnold, D.: A checkpoint compression study for high-performance computing systems. *The International Journal of High Performance Computing Applications* **29**(4), 387–402 (2015)
92. Hou, K.-Y., Shin, K.G., Turner, Y., Singhal, S.: Tradeoffs in compressing virtual machine checkpoints. In: *Proceedings of the 7th International Workshop on Virtualization Technologies in Distributed Computing*, pp. 41–48 (2013)
93. Repele, L., Muradore, R., Quaglia, D., Fiorini, P.: Improving performance of networked control systems by using adaptive buffering. *IEEE Trans. Ind. Electron.* **61**(9), 4847–4856 (2013)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.